Package ‘strategize’

June 10, 2026

Type Package

Title Tools for Learning Adversarial or Non-Adversarial Optimal Distributions in High-Dimensional Conjoint Experiments

Version 0.0.1

Date 2025-12-25

Author Connor T. Jerzak [aut, cre] (ORCID: <https://orcid.org/0000-0001-9858-2050>), Priyanshi Chandra [aut] (ORCID: <https://orcid.org/0009-0003-1625-8823>), Rishi Hazra [aut] (ORCID: <https://orcid.org/0000-0001-5018-4440>)

Maintainer Connor T. Jerzak <connor.jerzak@austin.utexas.edu>

Description The 'strategize' package implements methods for learning an optimal or adversarial probability distribution over high-dimensional factors in conjoint (or factorial) experiments. It supports both single-agent (non-adversarial) optimization and adversarial two-player settings, such as multi-stage electoral contexts. Users can estimate the distribution of factor levels that best achieves a chosen objective, optionally under institutional constraints (e.g., two-stage primaries). The package offers a variety of estimation routines, including closed-form solutions for some linear models, gradient-based optimizers for more complex outcome models, and built-in support for inference (via the delta method). It is particularly suitable for exploring and comparing candidate strategies in forced-choice conjoint studies, but is general enough for broader use in high-dimensional policy learning.

URL https://github.com/cjerzak/strategize-software

BugReports https://github.com/cjerzak/strategize-software/issues

Depends R (>= 4.1)

License GPL-3

Encoding UTF-8

Imports digest,
jsonlite,
stats,
compositions,
FactorHet,
ggplot2,
glinternet,
graphics,
gridExtra,
gtools,
Matrix,
matrixStats,

ps,
reticulate,
rlang,
rrapply,
sandwich,
utils

Suggests testthat (>= 3.0.0),
withr,
Rcpp,
devtools,
knitr,
rmarkdown,
tgp

VignetteBuilder knitr

RoxygenNote 7.3.3

Config/testthat/edition 3

NeedsCompilation no

Contents

`adapt_conjoint_foundation_model` *Adapt a pooled conjoint foundation model to a single study*

Description

Adapt a pooled conjoint foundation model to a single study

Usage

```
adapt_conjoint_foundation_model(
  foundation_model,
  Y,
  W,
  X = NULL,
  mode = c("auto", "pairwise", "single"),
  pair_id = NULL,
  profile_order = NULL,
  experiment_id = "adaptation_target",
  experiment_description = NULL,
  names_list = NULL,
  p_list = NULL,
  respondent_id = NULL,
  respondent_task_id = NULL,
  competing_group_variable_candidate = NULL,
  competing_group_variable_respondent = NULL,
  likelihood = NULL,
  n_outcomes = NULL,
  canonical_factor_id = NULL,
  canonical_level_id = NULL,
  neural_mcmc_control = NULL,
  foundation_adaptation_control = NULL,
  conda_env = "strategize_env",
  conda_env_required = TRUE,
  cache_path = NULL,
  cache_overwrite = FALSE,
  cache_compress = TRUE
)
```

Arguments

`foundation_model`
: A fitted `conjoint_foundation_model`.

`Y`
: Outcome vector.

`W`
: Factor matrix/data.frame.

`X`
: Optional numeric covariates.

`mode`
: `"auto"`, `"pairwise"`, or `"single"`.

`pair_id`
: Optional pair identifiers.

`profile_order`
: Optional within-pair ordering.

`experiment_id`
: Optional experiment identifier for the adaptation study.

`experiment_description`
: Optional text description for the adaptation study, used when text-aware experiment tokens are enabled.

`names_list`
: Optional factor level names.

`p_list`
: Optional `p_list`.

`respondent_id`
: Optional respondent identifiers.

`respondent_task_id`
: Optional respondent-task identifiers.

`competing_group_variable_candidate`
: Optional candidate competition-group labels for pairwise FM adaptation.

`competing_group_variable_respondent`
: Optional respondent competition-group labels for pairwise FM adaptation.

`likelihood`
: Optional likelihood override.

`n_outcomes`
: Optional categorical outcome count.

`canonical_factor_id`
: Optional factor-level sharing ids.

`canonical_level_id`
: Optional level-level sharing ids.

`neural_mcmc_control`
: Optional list passed to the Bayesian neural backend.

`foundation_adaptation_control`
: Optional list controlling adaptation. Supported keys are `strict_schema_match`, `allow_extra_covariates`, `use_text_semantics`, `text_embedding_fn`, `experiment_token_mode` `factor_tokenization`, `max_factor_tokens`, `covariate_value_encoding`, `shared_projection_` and `max_covariate_tokens`.

`conda_env`
: Conda env name for the neural backend.

`conda_env_required`
: Require the conda env to exist.

`cache_path`
: Optional predictor cache path.

`cache_overwrite`
: Logical; overwrite any existing cache at `cache_path`.

`cache_compress`
: Compression passed to `saveRDS()`.

Details

Current semantic zero-overlap foundation models are adapted canonically with `preference.fm::adapt_conjoint_fou`
This helper is retained for legacy runtime compatibility and errors when called on current semantic foundation objects. For legacy objects, it reuses the package's existing Bayesian neural outcome model rather than fitting a separate downstream architecture. Internally, this function:

1. normalizes the target study into the same local schema representation used by the neural outcome model,
2. finds the compatible foundation group matching `(mode, likelihood, n_outcomes)`,
3. builds warm-start values from the pooled foundation parameters, and
4. runs the current Bayesian neural fit with those warm starts.

Group matching is exact. Adaptation fails if the foundation object does not contain a compatible internal family for the requested target study.

The target-study data contract mirrors the per-experiment contract used by `fit_conjoint_foundation_model()`:

- `length(Y) == nrow(W)`;
- pairwise studies require `pair_id`;
- `X`, when supplied, must be numeric and row-aligned to `W`; raw numeric values remain unchanged during adaptation. Under the default FM covariate path, shared fused covariate/value tokens are aligned by pooled `X` name, optionally receive rebuilt `X`-name text embeddings, and are rebuilt from standardized numeric values plus local adaptation-study distribution summaries. Absent covariates are masked structurally rather than represented with a separate presence embedding;
- categorical adaptation follows the same `n_outcomes` rule as pooled training.

Schema reuse is partial by design:

- matched factors and levels inherit warm-started embeddings from the foundation fit;
- unmatched local schema elements fall back to local initialization;
- if `foundation_adaptation_control$strict_schema_match = TRUE`, unmatched factors cause an error instead of falling back.

The `foundation_adaptation_control` list supports:

`strict_schema_match` Require every local factor to match a pooled foundation slot. Default `FALSE`.

`allow_extra_covariates` Allow local covariates that were not present during pooled training. Extra local covariate tokens are appended after the shared pooled covariate vocabulary. Default `TRUE`.

`use_text_semantics` Reuse token-native text semantics during adaptation when the foundation object includes them, including factor-name, level-label, and pooled `X`-name token components. Default `TRUE`.

`text_embedding_fn` Optional embedding function used to rebuild token-native text information for the target study. When omitted, adaptation reuses the stored pooled text registry when available. Any supplied function must return the same embedding width as the pooled foundation text registry.

`experiment_token_mode` Experiment token mode for the adapted fit. Defaults to the pooled group's FM setting.

factor_tokenization Factor tokenizer used during adaptation. Defaults to the pooled group's FM setting.

max_factor_tokens Total factor-token budget used during adaptation. Defaults to the pooled group's FM setting.

covariate_value_encoding Covariate encoder family used during adaptation. Defaults to the pooled group's FM setting.

shared_projection_value_encoder Value-token generator used inside "shared_projection" during adaptation. Defaults to the pooled group's FM setting.

max_covariate_tokens Total covariate-token budget used during adaptation. Defaults to the pooled group's FM setting.

Value

A strategic_predictor.

See Also

fit_conjoint_foundation_model(), build_backend()

Examples

```
## Not run:
library(strategize)

build_backend(conda_env = "strategize_env")

text_embedding_fn <- function(x) {
  x <- as.character(x)
  cbind(nchar = nchar(x), has_space = as.numeric(grepl(" ", x)))
}

foundation_fit <- preference.fm::fit_conjoint_foundation_model(
  experiments = list(
    list(
      experiment_id = "study_a",
      experiment_description = "Source policy-message study",
      Y = c(1, 0, 0, 1),
      W = data.frame(
        price = c("Low", "Low", "High", "High"),
        message = c("Jobs", "Taxes", "Jobs", "Taxes")
      ),
      pair_id = c(1, 1, 2, 2),
      profile_order = c(1, 2, 1, 2),
      canonical_factor_id = c(price = "price", message = "message")
    )
  ),
  foundation_control = list(text_embedding_fn = text_embedding_fn)
)

adapted_fit <- preference.fm::adapt_conjoint_foundation_model(
  foundation_model = foundation_fit,
  Y = c(1, 0, 0, 1),
  W = data.frame(
    price = c("Low", "High", "Low", "High"),
    message = c("Jobs", "Jobs", "Taxes", "Taxes")
```

```
  ),
  mode = "pairwise",
  pair_id = c(1, 1, 2, 2),
  profile_order = c(1, 2, 1, 2),
  experiment_id = "target_study",
  experiment_description = "Target policy-message study",
  canonical_factor_id = c(price = "price", message = "message"),
  foundation_adaptation_control = list(text_embedding_fn = text_embedding_fn)
)

predict(
  adapted_fit,
  newdata = list(
    W = data.frame(
      price = c("Low", "High"),
      message = c("Jobs", "Taxes")
    ),
    pair_id = c(1, 1),
    profile_order = c(1, 2)
  )
)

## End(Not run)
```

`as_function` *Convert a fitted predictor to a scoring function*

Description

Convert a fitted predictor to a scoring function

Usage

```
as_function(fit)
```

Arguments

`fit`	A fitted `strategic_predictor`.

Value

A closure that calls `predict()` (or `predict_pair()` for pairwise).

build_backend — *Build the computational environment for* `strategize`

Description

Creates the conda environment used by the package's JAX-backed optimization workflow, neural APIs, and foundation checkpoint loading. Users may also create and manage a compatible environment themselves.

Usage

```
build_backend(
  conda_env = "strategize_env",
  conda = "auto",
  backend = c("auto", "cpu", "cuda", "mps"),
  force_reinstall = FALSE,
  text_embeddings = NULL,
  text_embedding_runtime = "auto",
  text_embedding_conda_env = "strategize_torch_env",
  verbose = TRUE
)
```

Arguments

- `conda_env` — Name of the conda environment in which to place the backends. Defaults to `"strategize_env"`.
- `conda` — The path to a conda executable. Using `"auto"` allows reticulate to attempt to automatically find an appropriate conda binary. Defaults to `"auto"`. If creation fails and a conda binary is found on PATH, the function retries with it.
- `backend` — JAX backend selector. Use `"auto"` to preserve the default host-aware behavior, `"cpu"` to install plain CPU JAX, `"cuda"` to require Linux and install CUDA-capable JAX wheels when supported by the NVIDIA driver, or `"mps"` to opt in to the experimental Apple Silicon `jax-mps` backend.
- `force_reinstall` — Logical. If `TRUE`, remove the named conda environment if it exists, then recreate and reinstall the backend from the requested input specifications.
- `text_embeddings` — Optional text embedding profile to install after the core backend is ready. Use `NULL` or `FALSE` for no text runtime, a profile name such as `"portable"`, `"harrier_oss_v1_0.6b_1024`, or `"qwen3_8b_4096"`, or a list with `profile`, `runtime`, and optional `model_id`.
- `text_embedding_runtime` — Runtime preference for `text_embeddings`; one of `"auto"`, `"mlx"`, `"cuda"`, `"rocm"`, or `"cpu"`.
- `text_embedding_conda_env` — Name of the conda environment used for optional text embedding runtimes. Defaults to `"strategize_torch_env"` so Torch/sentence-transformers packages do not share the JAX backend environment. Set to `conda_env` to preserve legacy single-environment behavior.
- `verbose` — Logical; if `TRUE`, print install milestones and stream long-running pip/conda output.

Details

The default backend = "auto" preserves the existing non-MPS behavior. The "mps" backend is experimental, opt-in, and supported only on macOS Apple Silicon. It creates a Python 3.13 environment, installs jax-mps, verifies that JAX reports the MPS backend under JAX_PLATFORMS=mps, and recreates an existing incompatible environment.

Value

Invisibly returns NULL. This function is called for its side effects of creating and configuring a conda environment for strategize. This function requires an Internet connection. You can find a list of conda Python paths via: Sys.which("python")

Examples

```
## Not run:
# Create a conda environment named "strategize_env"
# and install the required Python packages (jax, numpy, orbax-checkpoint, etc.)
build_backend(conda_env = "strategize_env", conda = "auto")

# If you want to specify a particular conda path:
# build_backend(conda_env = "strategize_env", conda = "/usr/local/bin/conda")

# Experimental Apple Silicon MPS backend:
# build_backend(backend = "mps")
# Sys.setenv(JAX_PLATFORMS = "mps") may be needed before importing JAX
# directly through reticulate in an already-running R session.

# Install the host-specific text embedding runtime for a profile:
# build_backend(text_embeddings = "harrier_oss_v1_0.6b_1024")
# This uses "strategize_torch_env" for Torch/sentence-transformers by default.

# Keep scripted runs quiet:
# build_backend(text_embeddings = "harrier_oss_v1_0.6b_1024", verbose = FALSE)

## End(Not run)
```

---

build_text_embedding_backend

Build a host-aware text-embedding backend

Description

Build a host-aware text-embedding backend

Usage

```
build_text_embedding_backend(
  conda_env = "strategize_torch_env",
  family = NULL,
  runtime = "auto",
  profile = "portable",
  model_id = NULL,
```

```
  cache_dir = NULL,
  cache_only = FALSE,
  batch_size = 64L,
  required = TRUE,
  verbose = TRUE,
  conda = "auto"
)
```

Arguments

conda_env	Conda env name used for the Python runtime. Defaults to "strategize_torch_env" so text embedding packages can be isolated from the JAX backend environment.
family	Optional embedding-family selector. When NULL, the selected profile supplies the family.
runtime	Runtime preference. Use "auto" to resolve per host, or choose one of "mlx", "cuda", "rocm", or "cpu" explicitly.
profile	Embedding profile. Use "portable" for the default portable profile or a registered profile such as "harrier_oss_v1_0.6b_1024" or "qwen3_8b_4096".
model_id	Optional model id override applied to every compatible candidate in the selected profile.
cache_dir	Optional directory for cached text embeddings.
cache_only	Logical; if TRUE, missing cache entries error before a model runtime is imported.
batch_size	Number of texts to encode per model call.
required	Logical; if TRUE, ensure the selected runtime is usable immediately. If FALSE, return a lazy backend that validates on first use.
verbose	Logical; if TRUE, print install milestones and stream long-running pip/conda output.
conda	Conda binary to use. Defaults to "auto".

Value

A list with $fn, $spec, and $runtime.

Examples

```
## Not run:
backend <- build_text_embedding_backend(conda_env = "strategize_torch_env")

study_a$experiment_description <- "Source study A"
study_b$experiment_description <- "Source study B"

foundation_fit <- preference.fm::fit_conjoint_foundation_model(
  experiments = list(study_a, study_b),
  foundation_control = list(
    text_embedding_fn = backend$fn
  )
)

inspect_text_embedding_backend(conda_env = "strategize_torch_env")

## End(Not run)
```

`check_hessian_geometry` *Check Hessian Geometry at Nash Equilibrium*

Description

Computes Hessian eigenvalues to verify proper saddle point structure at the Nash equilibrium. For a valid Nash equilibrium in a zero-sum game:

- AST (maximizer) Hessian should be negative semi-definite (all eigenvalues <= 0)
- DAG (minimizer) Hessian should be positive semi-definite (all eigenvalues >= 0)

Usage

```
check_hessian_geometry(result, tol = 1e-06, verbose = TRUE)
```

Arguments

`result`	Output from `strategize` with `adversarial = TRUE`
`tol`	Numeric. Tolerance for near-zero eigenvalues (default 1e-6)
`verbose`	Logical. Whether to print progress messages. Default is TRUE.

Details

Mathematical Foundation

At a Nash equilibrium in a zero-sum game, the Hessian matrices encode local curvature information:

- **AST (maximizes Q)**: The Hessian should be negative semi-definite, meaning all eigenvalues are <= 0. This confirms AST is at a local maximum.
- **DAG (minimizes Q)**: The Hessian should be positive semi-definite, meaning all eigenvalues are >= 0. This confirms DAG is at a local minimum.

The condition number (ratio of largest to smallest eigenvalue magnitude) indicates equilibrium robustness. High condition numbers suggest the equilibrium is sensitive to small perturbations.

Interpretation

- `status = "PASS"`: Valid saddle point with no flat directions
- `status = "WARNING"`: Valid saddle point but has flat directions (weak identification). Consider increasing regularization.
- `status = "FAIL"`: Not a proper saddle point. The solution may not have converged to a true Nash equilibrium.

Value

A list of class `"hessian_analysis"` containing:

status Character: "PASS", "WARNING", or "FAIL"

valid_saddle Logical. TRUE if both Hessians have correct definiteness

eigenvalues_ast Eigenvalues of AST player's Hessian

eigenvalues_dag Eigenvalues of DAG player's Hessian

is_negative_semidefinite_ast Logical. TRUE if AST Hessian is negative semi-definite

is_positive_semidefinite_dag Logical. TRUE if DAG Hessian is positive semi-definite

condition_number_ast Condition number of AST Hessian (stability indicator)

condition_number_dag Condition number of DAG Hessian

flat_directions_ast Number of near-zero eigenvalues (weak identification)

flat_directions_dag Number of near-zero eigenvalues

interpretation Character string with human-readable interpretation

Returns NULL with a warning if Hessian functions are not available.

See Also

`validate_equilibrium` for best-response validation

Examples

```
## Not run:
# Run adversarial strategize
result <- strategize(Y = y, W = w, adversarial = TRUE, nSGD = 500)

# Check Hessian geometry
hess <- check_hessian_geometry(result)
print(hess)

# Check if it's a valid saddle point
if (hess$valid_saddle) {
  message("Valid Nash equilibrium geometry!")
}

## End(Not run)
```

`conjoint-foundation` *Conjoint Foundation Models*

Description

Load pooled conjoint foundation-model artifacts produced by **preference.fm**.

Details

Foundation-model work now follows a split workflow:

1. Train pooled foundation models with `preference.fm::fit_conjoint_foundation_model()`.
2. Save them as checkpoint directories with `preference.fm::save_conjoint_foundation_bundle()`.
3. Load bundles, extract embeddings, and run predictors with **strategize**; adapt semantic foundation models with `preference.fm::adapt_conjoint_foundation_model()`.

Pooled training does not force every experiment into one universal output head. Experiments are grouped into compatible neural families defined by `(mode, likelihood, n_outcomes)`. Each compatible family shares one pooled schema-aware encoder and one neural fit, while incompatible families are stored as separate internal groups in the returned `conjoint_foundation_model`.

Cross-study sharing is driven by explicit canonical ids. Raw label equality alone does not force two studies to share factor or level identities. Optional text embeddings now enter the FM encoder directly: each candidate attribute is represented by one fused factor/value token, and each respondent covariate is represented by one fused covariate/value token. Factor and level information is fused by a two-layer SwiGLU MLP. Canonical ids still determine schema sharing across studies.

`fit_conjoint_foundation_model()` and `save_conjoint_foundation_bundle()` are retained as migration stubs that point to **preference.fm**. Use `preference.fm::adapt_conjoint_foundation_model()` for semantic target-study adaptation.

`country_latlong_cache` *Country latitude/longitude cache*

Description

Return the bundled country lookup table used by foundation-model place context tokens. The cache contains ISO codes, country names and aliases, and representative latitude/longitude values.

Usage

```
country_latlong_cache()
```

Value

A data frame with one row per country.

`create_p_list` *Create Baseline Probability List from Data*

Description

Generates a `p_list` suitable for the `strategize()` function from observed data or using uniform probabilities.

Usage

```
create_p_list(W, uniform = FALSE)
```

Arguments

`W`	Factor matrix (data.frame or matrix) with one column per conjoint factor.
`uniform`	Logical; if `TRUE`, use uniform probabilities across levels; if `FALSE` (default), use observed frequencies from the data.

Details

For conjoint experiments with balanced randomization, use `uniform = TRUE`. For experiments with intentional imbalance or to match observed frequencies, use `uniform = FALSE`.

Value

Named list where each element is a named numeric vector of probabilities. Names correspond to factor levels.

Examples

```
# Create sample factor matrix
W <- data.frame(
  Gender = c("Male", "Female", "Male", "Female"),
  Age = c("Young", "Old", "Young", "Old")
)

# Uniform probabilities (for balanced designs)
p_list_uniform <- create_p_list(W, uniform = TRUE)
print(p_list_uniform)
# $Gender
#   Male Female
#    0.5    0.5
# $Age
#   Young    Old
#     0.5    0.5

# Observed frequencies
p_list_observed <- create_p_list(W, uniform = FALSE)
print(p_list_observed)
```

`cv_strategize` *Cross-validation for Optimal Stochastic Interventions in Conjoint Analysis*

Description

Performs cross-validation to select the regularization parameter λ (and, if desired, other hyperparameters) for the `strategize` function. This function splits the data by respondent (or user-specified units), trains candidate models under a grid of λ values, and evaluates out-of-sample performance, returning the model that maximizes a chosen criterion (e.g., out-of-sample expected utility or log-likelihood).

Usage

```
cv_strategize(
  Y,
  W,
  X = NULL,
  lambda_seq = NULL,
  lambda = NULL,
```

```
  folds = 2L,
  crossfit_q = FALSE,
  crossfit_q_control = NULL,
  varcov_cluster_variable = NULL,
  competing_group_variable_respondent = NULL,
  competing_group_variable_candidate = NULL,
  competing_group_competition_variable_candidate = NULL,
  pair_id = NULL,
  respondent_id = NULL,
  respondent_task_id = NULL,
  profile_order = NULL,
  p_list = NULL,
  slate_list = NULL,
  use_optax = F,
  K = 1,
  nSGD = 100,
  diff = F,
  adversarial = F,
  adversarial_model_strategy = "four",
  partial_pooling = NULL,
  partial_pooling_strength = 50,
  use_regularization = TRUE,
  force_gaussian = F,
  force_reinforce = FALSE,
  temperature = NULL,
  a_init_sd = 0.001,
  learning_rate_max = 0.001,
  penalty_type = "KL",
  outcome_model_type = "glm",
  neural_mcmc_control = NULL,
  compute_se = T,
  conda_env = "strategize_env",
  conda_env_required = F,
  conf_level = 0.9,
  nFolds_glm = 3L,
  nMonte_adversarial = 5L,
  primary_pushforward = "mc",
  primary_strength = 1,
  primary_n_entrants = 1L,
  primary_n_field = 1L,
  nMonte_Qglm = 100L,
  optim_type = "gd",
  optimism = "extragrad",
  optimism_coef = 1,
  rain_lambda = 1,
  rain_gamma = 0.01,
  rain_L = NULL,
  rain_eta = 0.001,
  rain_variant = "alg10_staged",
  rain_output = "last"
)
```

Arguments

Y
: A numeric or binary response vector. If binary (e.g., 0–1), it should correspond to forced-choice outcomes (1 if candidate A is chosen; 0 if candidate B is chosen). If numeric, please see details in strategize for how outcomes are handled.

W
: A data frame or matrix representing the randomized conjoint attributes. Each column is a factor or character vector indicating attribute levels for a particular dimension. Multiple columns can be used if the conjoint has multiple attributes.

X
: Optional covariate matrix or data frame for modeling systematic heterogeneity. If K > 1, this is typically required for multi-class or cluster-based models. Otherwise, set X = NULL.

lambda_seq
: A numeric vector of candidate λ values for cross-validation. If NULL and lambda is also NULL, a sequence of values is automatically generated (e.g., via 10^seq(-4, 0, length.out = 5) * sd(Y)).

lambda
: A single user-specified λ value. If provided, cross-validation is effectively disabled unless lambda_seq is also supplied.

folds
: Either an integer number of cross-validation folds, a vector with one fold ID per observation, or a vector with one fold ID per unique respondent-task in first-seen order. Defaults to 2. Observation-level fold vectors must assign all rows from the same respondent-task to one fold.

crossfit_q
: Logical. If TRUE, compute Q_crossfit, Q_reference_crossfit, Q_gain_crossfit, and Q_crossfit_info on the final refit after lambda selection. Supported for non-adversarial pairwise average-case binomial GLM runs and adversarial pairwise binomial GLM runs with adversarial_model_strategy = "four".

crossfit_q_control
: Optional list passed to strategize to control cross-fitted Q evaluation, including the optional adversarial perspective_group entry.

varcov_cluster_variable
: An optional clustering variable for robust standard errors. For instance, if the data is from multiple respondents, specify respondent IDs here for cluster-robust inference (via sandwich estimation). If NULL, no cluster-based variance correction is used.

competing_group_variable_respondent
: Optional vector for multi-round or multi-group setups, indicating which respondent belongs to which group. Used for advanced or adversarial designs (e.g., dual-party contexts). If NULL, standard usage is assumed.

competing_group_variable_candidate
: Similar to competing_group_variable_respondent, but for candidate-level grouping. If NULL, standard usage is assumed.

competing_group_competition_variable_candidate
: An optional variable for specifying which candidate is in competition with which group. Relevant if multi-step adversarial frameworks are used.

pair_id
: An optional vector (same length as Y) identifying which rows (candidate pairs) belong to the same forced choice. For example, if each respondent evaluates multiple pairs, this ID ensures correct grouping. Required only in certain advanced difference-in-differences or paired analyses.

respondent_id
: A user-specified ID to denote respondent-level grouping, typically used to cluster standard errors or to perform out-of-sample validation by respondent. If NULL, a simple row index is used for splitting.

`respondent_task_id`
: Another optional ID for tasks (e.g., each respondent might see multiple tasks). Helps in advanced designs. If `NULL`, ignored.

`profile_order`
: An optional vector capturing the ordering of candidate profiles within tasks, if multiple profiles are being shown. Used in difference or extended hierarchical modeling.

`p_list`
: A list of assignment probabilities for each attribute, if known or desired as a baseline. If `NULL`, each level is assumed to have uniform probability or derived from empirical frequencies in `W`.

`slate_list`
: An optional list specifying alternative or restricted sets of attribute levels. Used when a subset of attributes is feasible or when bounding certain strategies in an adversarial design.

`use_optax`
: Logical. If `TRUE`, uses the **optax** Python library (via **reticulate**) for gradient-based optimization. If `FALSE`, uses a default gradient-based approach from **jax**.

`K`
: An integer specifying the number of mixture components or clusters if `X` is used (e.g., for multi-class analysis). Defaults to 1 (no mixture).

`nSGD`
: An integer number of iterations for gradient-based training. Defaults to 100 but can be increased if convergence has not been reached.

`diff`
: Logical indicating whether a difference-based model (e.g., for forced-choice or difference-in-outcomes) is used. Defaults to `FALSE`, but set `TRUE` in certain difference-of-utility designs.

`adversarial`
: Logical indicating whether to use a two-party or multi-agent *adversarial* approach in the optimization. If `TRUE`, a min-max (zero-sum) formulation is employed. Defaults to `FALSE` (single-agent or average-case optimization).

`adversarial_model_strategy`
: Character string indicating whether to estimate `"four"` outcome models (primary + general for each group), `"two"` outcome models (one per group reused for both primary and general), or `"neural"` (Bayesian Transformer models with party tokens; defaults to a single pooled model across groups and stages. Set `neural_mcmc_control$n_bayesian_models = 2` to fit separate AST/DAG models) in adversarial mode.

`partial_pooling`
: Logical indicating whether to partially pool (shrink) group-specific outcome model coefficients toward a shared average when using the "two" strategy. When `NULL`, defaults to `TRUE` in the two-strategy adversarial case.

`partial_pooling_strength`
: Numeric scalar controlling the amount of shrinkage used for partial pooling in the two-strategy adversarial case.

`use_regularization`
: Logical; if `TRUE`, penalty-based regularization is used for the outcome model. Usually set to `TRUE` for large designs. Defaults to `TRUE`.

`force_gaussian`
: Logical indicating whether a Gaussian family (`lm`-style) is forced for the outcome model, even if `Y` is binary. Defaults to `FALSE`.

`force_reinforce`
: Logical indicating whether to force REINFORCE-based optimization for the neural average-case objective when exact small-support enumeration is available. This affects the optimization objective only; final reported `Q` values still use the standard report-time evaluation path. Defaults to `FALSE`.

`temperature` Optional numeric temperature controlling the smoothness of Gumbel-Softmax sampling when exploring probability vectors. Smaller values lead to distributions closer to the argmax. Defaults to `NULL`, which allows internal defaults.

`a_init_sd` A numeric controlling the random initialization scale for unconstrained parameters in gradient-based optimization. Defaults to 0.001. Larger values can help avoid local minima in complex outcome landscapes.

`learning_rate_max`
Base learning rate for gradient-based optimizers. Defaults to `0.001`.

`penalty_type` A character string specifying the type of penalty for the *optimal stochastic intervention*, e.g., `"KL"`, `"L2"`, or `"LogMaxProb"`. The default is `"KL"`.

`outcome_model_type`
Character string indicating the outcome model to use, such as `"glm"` for generalized linear models or `"neural"` for a neural-network approximation. Defaults to `"glm"`.

`neural_mcmc_control`
Optional list overriding default MCMC settings used when `outcome_model_type = "neural"`. Use `neural_mcmc_control$uncertainty_scope = "output"` to restrict delta-method uncertainty to output-layer parameters. In adversarial neural mode, set `neural_mcmc_control$n_bayesian_models = 2` to fit separate AST/DAG models (default is 1 for a single differential model). Use `neural_mcmc_control$ModelDi` and `neural_mcmc_control$ModelDepth` to override the Transformer hidden width and depth. Pairwise neural fits default to `neural_mcmc_control$cross_candidate_encoder = "term"` when unspecified, except rank-positive low-rank respondent-candidate interaction defaults to `"none"` to avoid duplicating pairwise interaction capacity. Set `neural_mcmc_control$cross_candidate_encoder = "term"` (or `TRUE`) to include the opponent-dependent cross-candidate term in pairwise mode, set `neural_mcmc_control$c = "attn"` to add a lightweight cross-attention step. When combined with `neural_mcmc_control$res = "full_attn"`, that step consumes the depth-attended candidate-token readout. Set `neural_mcmc_control$cross_candidate_encoder = "full"` to enable a full cross-encoder that jointly encodes both candidates. Use `"none"` (or `FALSE`) to disable. Set `neural_mcmc_control$residual_mode = "full_attn"` to replace the default additive/ReZero residual path with full depth-wise attention over all prior layer outputs plus a final depth-attentive readout; use `"standard"` (default) to keep the existing residual formulation. Rank-positive low-rank Bernoulli pairwise logits use RMS logit normalization by default; set `neural_mcmc_control$low_rank_logit = "none"` to disable it. Explicit `low_rank_logit_transform = "softclip"` remains supported for compatibility. For variational inference (subsample_method = "batch_vi"), set `neural_mcmc_control$optimizer` to `"muon"` (default when `optax.contrib.muon` is available), `"adam"` (numpyro.optim), `"adamw"` (AdamW), or `"adabelief"` (optax). Muon is only applied to matrix-valued model-weight locations when the guide preserves matrix structure; with `vi_guide = "auto_diagonal"`, it falls back to AdamW/Adam. Learning-rate decay is controlled by `neural_mcmc_control$svi_ste` (integer steps, or `"optimal"` for a scaling-law heuristic based on model/data size; for minibatched VI this also scales with `batch_size`) and `neural_mcmc_control$svi_lr_sche` (default `"warmup_cosine"`), with optional `svi_lr_warmup_frac` and `svi_lr_end_factor`. Validation-based early stopping is enabled by default for SVI-backed neural fits; set `neural_mcmc_control$early_stopping = FALSE` to force the full `svi_steps` budget. Use `neural_mcmc_control$early_stopping_n_checks` (default `10`) to request an approximate number of validation checks during SVI early stopping; the cadence is derived as `ceiling(svi_steps / early_stopping_n_checks)`. Use `neural_mcmc_control$early_stopping_patience` (default `3`) to control how many consecutive non-improving validation checks are tolerated before

stopping. For compact streaming SVI, early_stopping = TRUE enables validation best-checkpoint selection but still runs the full svi_steps budget. Set neural_mcmc_control$early_stopping_validation_frac (default 0.05) to retain approximately that fraction of evaluable observations in the held-out validation split, and optionally neural_mcmc_control$early_stopping_validation_max_n (default 2048) to cap the retained validation size after fraction-based sizing. Set early_stopping_validation_max_n = NULL to disable that cap. Checkpoint gradient diagnostics are enabled by default for SVI fits and are evaluated at the early-stopping checkpoint cadence; set neural_mcmc_control$gradient_diagnostics = FALSE to disable them.

compute_se
: Logical; if TRUE, attempts to compute standard errors using M-estimation or the Delta method. Defaults to TRUE.

conda_env
: A character specifying the name of a Conda environment for the JAX-backed optimization workflow. Defaults to "strategize_env".

conda_env_required
: Logical. If TRUE, errors if the specified Conda environment conda_env cannot be found. Otherwise tries to fall back gracefully.

conf_level
: The confidence level (between 0 and 1) for interval estimation, default 0.90.

nFolds_glm
: An integer specifying the number of folds in internal regression-based cross-validation (if used) for outcome model selection. Defaults to 3.

nMonte_adversarial
: A positive integer specifying the number of Monte Carlo draws for the min-max (adversarial) stage, if adversarial = TRUE. Defaults to 5.

primary_pushforward
: Character string controlling the primary-stage push-forward estimator. Use "mc" (default) for Monte Carlo sampling with per-draw primary winners, or "linearized" for the faster averaged-weight approximation, or "multi" for multi-candidate primaries.

primary_strength
: Numeric scalar controlling primary decisiveness (see strategize).

primary_n_entrants
: Integer number of entrant candidates per party in multi-candidate primaries.

primary_n_field
: Integer number of field candidates per party in multi-candidate primaries.

nMonte_Qglm
: An integer specifying the number of Monte Carlo draws for evaluating certain integrals in glm-based approximations, default 100.

optim_type
: A character describing the optimization routine. Typically "default" uses a standard gradient-based approach; set "tryboth" or "SecondOrder" for testing or advanced routines.

optimism
: Character string controlling optimistic / extra-gradient updates for the gradient optimizer. Options: "extragrad" (default), "smp" (stochastic mirror-prox), "ogda", "rain" (RAIN: Recursive Anchored Iteration with anchored extragradient and increasing quadratic anchor penalties), or "none". Only supported when use_optax = FALSE.

optimism_coef
: Numeric scalar controlling the magnitude of optimism adjustments. For "ogda", this scales the optimistic correction term. Ignored when optimism = "rain".

rain_lambda
: Numeric scalar giving the base RAIN regularization scale λ. The staged algorithm uses $\lambda_0 = \gamma\lambda$ and grows by $(1+\gamma)$ each stage. Only used when optimism = "rain".

`rain_gamma` Non-negative numeric scalar for the RAIN anchor-growth parameter γ. If not supplied, the default is auto-scaled downward when `nSGD` exceeds 100 to keep total anchor growth roughly constant. Only used when `optimism = "rain"`.

`rain_L` Optional numeric scalar for the Lipschitz constant L used by RAIN. When supplied and `rain_eta` is missing, `rain_eta` defaults to $1/(8L)$ and stage lengths follow $T_s = 16L/\lambda_s$. If `NULL`, L is estimated from `rain_eta`. Only used when `optimism = "rain"`.

`rain_eta` Optional numeric scalar step size η for RAIN. Defaults to `0.001` and is auto-scaled downward when `nSGD` exceeds 100 if not supplied. If `rain_L` is supplied and `rain_eta` is missing, defaults to $1/(8L)$. Only used when `optimism = "rain"`.

`rain_variant` Character string specifying the RAIN variant. Options: `"alg10_staged"` (merged Algorithm 10 with recursive stage anchors; default) or `"alg9_single_loop"` (single-loop variant; not yet implemented). Only used when `optimism = "rain"`.

`rain_output` Character string controlling the stage output for RAIN. `"uniform_half"` samples uniformly from half-iterates within the stage (most faithful); `"last"` returns the last iterate (default). Only used when `optimism = "rain"`.

Details

`cv_strategize` implements a cross-validation routine for `strategize`. First, the data is split into `folds` parts. For each fold, we train candidate outcome models and compute out-of-sample performance. The best-performing λ is selected. Finally, a refit on the full data is done using the chosen hyperparameters, returning the results of the final `strategize` call with λ set to the best value.

The function supports a wide range of conjoints, including forced-choice (where `diff = TRUE`), multi-cluster outcome modeling (where $K > 1$), and adversarial designs (where `adversarial = TRUE`). Regularization for the outcome model or for the candidate distribution can be enabled via `use_regularization` and `penalty_type`. Cross-validation is particularly helpful when the data is limited or highly dimensional.

Value

A named list with components:

pi_star_point The estimated optimal probability distribution(s) over candidate profiles ($\hat{\boldsymbol{\pi}}^*$).

Q_point The primary estimated expected outcome (e.g., vote share) under the selected optimal distribution.

Q_se The primary standard error for `Q_point`, when standard errors are requested.

Q_point_mEst, Q_se_mEst Backward-compatible aliases for `Q_point` and `Q_se`.

lambda The chosen λ value from cross-validation (and any other relevant hyperparameters).

CVInfo A data frame or matrix summarizing cross-validation results, e.g., in-sample and out-of-sample estimates for each candidate λ.

Q_crossfit, Q_reference_crossfit, Q_gain_crossfit, Q_crossfit_info Optional final-refit cross-fitted policy-value diagnostics when `crossfit_q = TRUE`.

Other components Various additional objects useful for inference and debugging (e.g., final model fits, standard error estimates, weighting details).

See Also

`strategize` for direct optimization of stochastic interventions in conjoint analysis, including average and adversarial settings.

Examples

```
# ================================================
# Cross-validation to select regularization lambda
# ================================================
set.seed(123)
n <- 400  # profiles (200 pairs)

# Generate factor matrix
W <- data.frame(
  Gender = sample(c("Male", "Female"), n, replace = TRUE),
  Age = sample(c("35", "50", "65"), n, replace = TRUE),
  Party = sample(c("Dem", "Rep"), n, replace = TRUE)
)

# Simulate outcome with true effects
latent <- 0.2 * (W$Gender == "Female") + 0.15 * (W$Age == "35")
prob <- plogis(latent)

# Create paired forced-choice structure
pair_id <- rep(1:(n/2), each = 2)
Y <- numeric(n)
for (p in unique(pair_id)) {
  idx <- which(pair_id == p)
  winner <- sample(idx, 1, prob = prob[idx])
  Y[idx] <- as.integer(seq_along(idx) == which(idx == winner))
}
profile_order <- rep(1:2, n/2)

# Cross-validate over lambda values
# Lower lambda = less regularization = further from baseline
cv_result <- cv_strategize(
  Y = Y,
  W = W,
  lambda_seq = c(0.01, 0.1, 0.5, 1.0),
  folds = 2,
  pair_id = pair_id,
  respondent_id = pair_id,
  profile_order = profile_order,
  diff = TRUE,
  nSGD = 50,
  compute_se = FALSE
)

# View CV results and selected lambda
print(cv_result$lambda)       # Optimal lambda
print(cv_result$CVInfo)       # Performance at each lambda
print(cv_result$pi_star_point) # Optimal distribution
print(cv_result$Q_point)       # Expected outcome
```

`extract_embeddings` *Extract last-layer embeddings from fitted neural objects*

Description

Extract the final hidden representation immediately before the neural output head, or the model's respondent-context representation.

Usage

```
extract_embeddings(object, ...)

## S3 method for class 'strategic_predictor'
extract_embeddings(
  object,
  newdata,
  level = c("candidate", "respondent_context", "all"),
  factor_schema = NULL,
  text_embedding_fn = NULL,
  ...
)

## S3 method for class 'conjoint_foundation_model'
extract_embeddings(
  object,
  newdata,
  group_key = NULL,
  mode = c("auto", "pairwise", "single"),
  likelihood = NULL,
  n_outcomes = NULL,
  level = c("candidate", "respondent_context", "all"),
  experiment_id = NULL,
  names_list = NULL,
  p_list = NULL,
  canonical_factor_id = NULL,
  canonical_level_id = NULL,
  foundation_adaptation_control = NULL,
  conda_env = "strategize_env",
  conda_env_required = TRUE,
  ...
)
```

Arguments

`object`	A fitted neural object. Supported classes are `strategic_predictor` and `conjoint_foundation_moc`
`...`	Additional method-specific arguments.
`newdata`	New data. For `strategic_predictor`, this follows the same contract as `predict()`. For `conjoint_foundation_model`, use either a list containing `W` and optional `X`, `pair_id`, `profile_order`, and `competing_group_variable_candidate`, `competing_group_variable_respondent`, and `experiment_id`, or a data frame/matrix. When supplying a raw data frame to the foundation method, extra numeric `X` columns can only be separated from factor columns when `names_list` (or `p_list`) identifies the factor columns.
`group_key`	Optional internal foundation group key to extract from.
`mode`	Optional mode filter used for foundation-group selection.

likelihood	Optional likelihood filter used for foundation-group selection.
n_outcomes	Optional outcome-count filter used for foundation-group selection.
level	Embedding level to return. "candidate" preserves the historical candidate/profile output, "respondent_context" returns one respondent-context matrix, and "all" returns both.
factor_schema	Optional explicit prediction-time factor schema for fused neural predictors. Supply a list with names_list or p_list, and optionally precomputed factor/level text or structural matrices.
text_embedding_fn	Optional text embedding function used with factor_schema and prediction-time experiment_description.
experiment_id	Optional experiment identifier used for experiment-token lookup.
names_list	Optional local factor-level metadata for raw foundation extraction.
p_list	Optional p_list used to derive names_list when needed.
canonical_factor_id	Optional canonical factor ids used to map local factors into a pooled foundation group.
canonical_level_id	Optional canonical level ids used to map local levels into a pooled foundation group.
foundation_adaptation_control	Optional list controlling schema matching for raw foundation extraction. Supported keys are strict_schema_match and allow_extra_covariates.
conda_env	Conda env name for neural extraction when params must be reconstructed.
conda_env_required	Require the conda env to exist.

Details

strategic_predictor extraction works on fitted neural predictors, including objects returned by strategic_prediction() and adapt_conjoint_foundation_model().

conjoint_foundation_model extraction runs directly on a selected pooled foundation group. When extracting from a raw foundation object, local studies must be mappable into the pooled schema, typically via matching experiment_id, canonical ids, or direct use of pooled slot names and level keys.

Respondent-context embeddings are derived from the model's context-side tokens: respondent group, respondent covariates, experiment tokens, and pairwise stage/matchup tokens when those components are available.

This function does not extend predict() or predict_pair(); it is a dedicated embedding API.

Value

An object of class strategic_embeddings.

For level = "candidate", single-mode models contain $embeddings, an $n \times d$ numeric matrix.

For pairwise models:

- non-"full" cross-candidate encoders return $left and $right matrices with one row per pair;

- "full" cross-candidate encoders return $joint, the pair-level final readout used immediately before the output head.

For level = "respondent_context", the return value contains $respondent_context, an $n \times d$ or pair-level $n_{pairs} \times d$ matrix. For level = "all", the candidate fields and $respondent_context are returned together.

All outputs also include $metadata, including the source class, mode, requested level, cross-candidate encoder mode, embedding width, and context component availability.

Examples

```
## Not run:
fit <- strategic_prediction(
  Y = y,
  W = w,
  model = "neural",
  mode = "single"
)

emb <- extract_embeddings(fit, newdata = w[1:10, , drop = FALSE])

foundation_emb <- extract_embeddings(
  foundation_fit,
  newdata = list(
    W = target_w,
    X = target_x,
    pair_id = target_pair_id,
    profile_order = target_profile_order
  ),
  group_key = "pairwise::bernoulli::1",
  canonical_factor_id = c(price = "price", message = "message")
)

## End(Not run)
```

---

fit_conjoint_foundation_model

Pooled conjoint foundation-model training moved to preference.fm

---

Description

Pooled conjoint foundation-model training moved to preference.fm

Usage

```
fit_conjoint_foundation_model(
  experiments,
  foundation_control = NULL,
  conda_env = "strategize_env",
  conda_env_required = TRUE,
  cache_path = NULL,
  cache_overwrite = FALSE,
  cache_compress = TRUE
)
```

Arguments

`experiments` List of experiment specifications. Each element must be a named list with at least `experiment_id`, `Y`, and `W`.

`foundation_control` Optional list controlling pooled training. Supported keys are `add_experiment_indicators`, `add_text_semantics`, `text_embedding_fn`, and `neural_mcmc_control`.

`conda_env` Conda env name for the neural backend.

`conda_env_required` Require the conda env to exist.

`cache_path` Optional path to a cached foundation bundle.

`cache_overwrite` Logical; overwrite any existing cache at `cache_path`.

`cache_compress` Retained for API compatibility.

Details

This exported name is retained for migration guidance. Pooled training is now owned by **preference.fm**. Use `preference.fm::fit_conjoint_foundation_model()` to train pooled models, `preference.fm::adapt_conjoint_foundation_model()` for semantic adaptation, and **strategize** for loading, embedding extraction, and prediction.

Each element of `experiments` is a per-study specification with the following contract.

Required fields:

- `experiment_id` A unique study identifier.
- `Y` Outcome vector with `length(Y) == nrow(W)`.
- `W` A data frame or matrix of factor columns.

Optional fields:

- `mode` `"auto"`, `"pairwise"`, or `"single"`. Defaults to `"pairwise"` when `pair_id` is supplied and `"single"` otherwise.
- `pair_id` Required for pairwise studies. Each pair id must appear exactly twice.
- `profile_order` Optional within-pair ordering, typically `1/2`.
- `X` Optional numeric covariates aligned row-wise to `W`. Non-numeric columns are rejected. Raw numeric values remain unchanged at the API boundary. In the default FM covariate path (`covariate_value_encoding = "shared_projection"` with `shared_projection_value_encoder = "name_dist_moe"`), each present covariate is emitted as one fused covariate/value token. The fused token can include projected `X`-name text embeddings and combines the standardized numeric value with local study-specific distribution summaries and covariate-name semantics. Absent covariates are masked structurally rather than represented by a separate learned presence embedding.
- `respondent_id`, `respondent_task_id` Optional row-aligned respondent/task identifiers used when available for clustering and evaluation logic.
- `competing_group_variable_candidate`, `competing_group_variable_respondent` Optional row-aligned competition-group labels for pairwise studies. When both are supplied and the candidate labels span both same-group and cross-group pairs, the pooled FM trains in a stage-aware pairwise mode. Otherwise the pairwise FM remains stage-free.
- `likelihood` `"auto"`, `"bernoulli"`, `"categorical"`, or `"normal"`.

n_outcomes Required only when forcing a categorical likelihood. In v1 it must match the number of observed classes in the study. Categorical outcomes are internally normalized to zero-based class ids before neural fitting.

names_list, p_list Optional factor-level metadata used to define the local level universe for each factor.

canonical_factor_id Optional named vector or list that forces cross-study sharing of factor identities when explicitly provided.

canonical_level_id Optional named list that forces cross-study sharing of level identities when explicitly provided.

Pooled training groups experiments by backend-compatible task mode. Pairwise context mode is a supported capability of a pairwise group rather than a split key: stage-free and stage-aware pairwise studies can share one group. If any pooled pairwise study requires stage-aware context, the shared pairwise group is promoted to stage-aware-capable and records all supported pairwise context modes. The returned object may therefore hold multiple internal foundation groups when the input studies mix pairwise and single designs.

Schema sharing rules are conservative:

- explicit canonical ids force sharing;
- absent factors in a pooled schema are routed to holdout rows during training;
- raw text equality alone does not merge schema elements.

The foundation_control list supports:

add_experiment_indicators Whether to append experiment indicators. This field is retained for compatibility, but the foundation neural path now always represents experiment identity as an explicit experiment token rather than one-hot covariates.

add_text_semantics Whether to add token-native text semantics when text_embedding_fn is supplied. This covers projected factor-name embeddings, level-label embeddings, and pooled X-name embeddings inside the emitted tokens. Default TRUE.

text_embedding_fn Optional function that maps character input to numeric embeddings. It may accept a character vector and return a matrix with matching rows, or accept one string at a time and return a fixed-width numeric vector. These embeddings are projected into FM factor-name, factor-level, and pooled X-name token components. Canonical ids still govern cross-study pooling and schema sharing.

experiment_token_mode Experiment token mode for FM training. "description" uses experiment-description text, "hybrid" combines description text and learned pooled experiment ids, and "legacy_id" uses only learned pooled experiment ids. Default "description".

factor_tokenization FM factor tokenizer. The required "fused" mode emits one fused factor/value token per candidate attribute. Factor and level information is fused by a two-layer SwiGLU MLP.

max_factor_tokens Total factor-token budget used by the FM fused factor encoder. The default 256L supports up to 256 factor attributes per profile.

covariate_value_encoding FM covariate encoder family. The required "shared_projection" mode emits one fused covariate/value token per respondent covariate.

shared_projection_value_encoder Value-token generator used inside "shared_projection". The default "name_dist_moe" conditions each covariate value token on covariate-name semantics and local distribution summaries. "legacy_scalar" falls back to the older standardized scalar projection.

`max_covariate_tokens` Total covariate-token budget used by the FM fused covariate encoder. The default `512L` supports up to 512 covariates per row.

`neural_mcmc_control` Optional list passed to the existing neural outcome backend. Defaults to a pooled SVI configuration using output-only uncertainty.

Value

An object of class `conjoint_foundation_model`.

See Also

`adapt_conjoint_foundation_model()`, `save_conjoint_foundation_bundle()`, `load_conjoint_foundation_bun`
`build_backend()`

Examples

```
## Not run:
library(strategize)

build_backend(conda_env = "strategize_env")

text_embedding_fn <- function(x) {
  x <- as.character(x)
  cbind(nchar = nchar(x), has_space = as.numeric(grepl(" ", x)))
}

study_a <- list(
  experiment_id = "study_a",
  experiment_description = "Pairwise policy-message study",
  Y = c(1, 0, 0, 1),
  W = data.frame(
    price = c("Low", "Low", "High", "High"),
    message = c("Jobs", "Taxes", "Jobs", "Taxes")
  ),
  pair_id = c(1, 1, 2, 2),
  profile_order = c(1, 2, 1, 2),
  canonical_factor_id = c(price = "price", message = "message")
)

study_b <- list(
  experiment_id = "study_b",
  experiment_description = "Pairwise messenger study",
  Y = c(0, 1, 1, 0),
  W = data.frame(
    price = c("Low", "High", "Low", "High"),
    message = c("Jobs", "Jobs", "Taxes", "Taxes"),
    messenger = c("Local", "Local", "National", "National")
  ),
  pair_id = c(1, 1, 2, 2),
  profile_order = c(1, 2, 1, 2),
  canonical_factor_id = c(
    price = "price",
    message = "message",
    messenger = "messenger"
  )
)
```

```
foundation_fit <- preference.fm::fit_conjoint_foundation_model(
  experiments = list(study_a, study_b),
  foundation_control = list(
    text_embedding_fn = text_embedding_fn,
    neural_mcmc_control = list(
      ModelDims = 32L,
      ModelDepth = 1L,
      subsample_method = "batch_vi",
      uncertainty_scope = "output",
      svi_steps = 100L
    )
  )
)

## End(Not run)
```

get_final_gradients *Get Final Gradient Norms*

Description

Extract the final gradient magnitudes from a strategize result.

Usage

```
get_final_gradients(result)
```

Arguments

`result`	Output from `strategize`

Value

Named numeric vector with gradient norms for AST and DAG players.

helpers *User-Facing Helper Functions for strategize*

Description

Convenience functions to simplify common tasks when using the strategize package. These functions reduce the complexity of the main API by providing sensible defaults and automatic data processing.

`inspect_text_embedding_backend` *Inspect a host-aware text-embedding backend*

Description

Inspect a host-aware text-embedding backend

Usage

```
inspect_text_embedding_backend(
  conda_env = "strategize_torch_env",
  family = NULL,
  runtime = "auto",
  profile = "portable",
  model_id = NULL,
  conda = "auto"
)
```

Arguments

`conda_env`	Conda env name used for the Python runtime. Defaults to `"strategize_torch_env"` so text embedding packages can be isolated from the JAX backend environment.
`family`	Optional embedding-family selector. When `NULL`, the selected profile supplies the family.
`runtime`	Runtime preference. Use `"auto"` to resolve per host, or choose one of `"mlx"`, `"cuda"`, `"rocm"`, or `"cpu"` explicitly.
`profile`	Embedding profile. Use `"portable"` for the default portable profile or a registered profile such as `"harrier_oss_v1_0.6b_1024"` or `"qwen3_8b_4096"`.
`model_id`	Optional model id override applied to every compatible candidate in the selected profile.
`conda`	Conda binary to use. Defaults to `"auto"`.

Value

A serializable list describing the host, Python env health, candidate text-embedding runtimes, and the selected backend.

`is_adversarial` *Check if Result is Adversarial*

Description

Utility function to check if a strategize result is from adversarial mode.

Usage

```
is_adversarial(result)
```

Arguments

`result`	Output from `strategize`

Value

Logical. TRUE if the result is from adversarial mode.

`load_conjoint_foundation_bundle` *Load a conjoint foundation bundle*

Description

Load a conjoint foundation bundle

Usage

```
load_conjoint_foundation_bundle(
  file,
  conda_env = "strategize_env",
  conda_env_required = TRUE,
  preload_params = FALSE
)
```

Arguments

`file`	Path to a checkpoint directory created by `preference.fm::save_conjoint_foundation_bundle()` or to a legacy `.rds` bundle.
`conda_env`	Conda env name for the neural backend.
`conda_env_required`	Require the conda env to exist.
`preload_params`	Logical; reconstruct neural params immediately.

Details

Loading reconstructs a `conjoint_foundation_model` without importing or depending on **preference.fm**. New checkpoint directories restore direct neural parameter PyTrees from Orbax. Legacy `.rds` foundation bundles remain readable for older artifacts.

Value

A `conjoint_foundation_model`.

See Also

`save_conjoint_foundation_bundle()`

Examples

```
## Not run:
foundation_fit <- load_conjoint_foundation_bundle("foundation_bundle")

## End(Not run)
```

`load_neural_outcome_bundle` *Load a portable neural outcome bundle*

Description

Load a portable neural outcome bundle

Usage

```
load_neural_outcome_bundle(
  file,
  conda_env = "strategize_env",
  conda_env_required = TRUE,
  preload_params = FALSE
)
```

Arguments

`file`	Path to a bundle created by `save_neural_outcome_bundle()`.
`conda_env`	Conda env name for neural backend. Use `NULL` to defer to metadata.
`conda_env_required`	Require conda env to exist for neural backend.
`preload_params`	Logical; if TRUE, reconstruct neural params immediately.

Value

A `strategic_predictor` ready for `predict()` / `predict_pair()`.

`load_strategic_predictor` *Load a strategic predictor from disk*

Description

Load a strategic predictor from disk

Usage

```
load_strategic_predictor(
  file,
  conda_env = "strategize_env",
  conda_env_required = TRUE
)
```

Arguments

`file`	Path to a cached predictor created by `save_strategic_predictor()`.
`conda_env`	Conda env name for neural predictors. Use `NULL` to defer to the cached metadata.
`conda_env_required`	Require conda env to exist for neural predictors.

Value

A `strategic_predictor` ready for `predict()`.

`plot_best_response_curves` *Plot Dimension-by-Dimension Best-Response Curves from Adversarial* `strategize()` *Output*

Description

`plot_best_response_curves` takes the result of an adversarial `strategize` run (i.e., with `adversarial = TRUE`) and produces dimension-specific best-response curves. Specifically, for a chosen factor dimension d, it plots:

1. The curve of $\pi^*_{\mathrm{dag},d}$ as a function of $\pi_{\mathrm{ast},d}$.
2. The curve of $\pi^*_{\mathrm{ast},d}$ as a function of $\pi_{\mathrm{dag},d}$.

Potential intersection points in this 2D space can indicate approximate equilibria for dimension d, holding the other dimensions fixed at the solution found by `strategize`.

This function is computationally intensive: for each of `nPoints_br` grid values of $\pi_{\mathrm{ast},d}$, it searches over possible $\pi_{\mathrm{dag},d}$ (and vice versa) to find each side's best response through batched JAX grid evaluation. Nonetheless, it provides a direct visualization of how each player (ast or dag) responds to changes in the other's distribution along a single factor dimension.

Usage

```
plot_best_response_curves(
  res,
  d_ = 1,
  nPoints_br = 100L,
  nPoints_heat = 50L,
  title = NULL,
  col_ast = "blue",
  col_dag = "red",
  lwd_ast = 2,
  lwd_dag = 2,
  point_pch = 19,
  silent = FALSE
)
```

Arguments

`res` A list returned by strategize, which must include adversarial references. Internally, `res` should contain items like `res$a_i_ast`, `res$a_i_dag`, the JAX-based functions (`dQ_da_ast`, `dQ_da_dag`, `QFXN`, ...), along with the appropriate unconstrained parameter vectors.

`d_` (Integer) The dimension of $\pi_{\text{ast}}, \pi_{\text{dag}}$ to examine. For example, if you have multiple factors (dimensions), each is indexed by a positive integer. Defaults to 1.

`nPoints_br` (Integer) Number of equally spaced grid points in $[0, 1]$ to sample for *the outer* loop. The code does an internal small search for best responses at each grid point. Larger `nPoints_br` => smoother curves but more computation. Defaults to `100L`.

`nPoints_heat` (Integer) Number of grid points for heat map calculations. Defaults to `50L`.

`title` (Character or `NULL`) Main plot title. If `NULL`, an auto-generated title is used, e.g. "Best-Response Curves (Dimension d_=1)".

`col_ast, col_dag` (Character) Colors for ast's and dag's best-response curves, respectively. Defaults to `"blue"` (ast) and `"red"` (dag).

`lwd_ast, lwd_dag` (Numeric) Line widths for ast and dag curves, respectively. Default is 2.

`point_pch` (Numeric) Symbol for marking the approximate intersection (if found) on the plot. Defaults to 19 (filled circle).

`silent` (Logical) If `TRUE`, suppresses printed messages during the search for intersection. Defaults to `FALSE`.

Details

Mechanics: For each $\pi_{\text{ast},d} \in \{0, \frac{1}{nPoints_b r-1}, \ldots, 1\}$, the function temporarily fixes that dimension in the ast player's unconstrained parameter vector. It then does an internal grid search over $\pi_{\text{dag},d}$ to see which value yields the largest dag payoff (lowest ast payoff), consistent with `adversarial=TRUE`. The resulting curve is $\text{BR}_{\text{dag}}(\pi_{\text{ast},d})$.

Likewise, it holds $\pi_{\text{dag},d}$ fixed and searches over $\pi_{\text{ast},d}$ to get $\text{BR}_{\text{ast}}(\pi_{\text{dag},d})$.

The intersection in $(\pi_{\text{ast},d}, \pi_{\text{dag},d})$-space (if one exists in the discretized grid) is a candidate local equilibrium for that factor dimension, *given that the other factor dimensions remain at the solution from* strategize.

Performance Caution: This batched grid search evaluates many candidate responses, which may still be slow for large `nPoints_br` or complex outcome models. Consider using a smaller `nPoints_br` if performance is an issue, or focusing on only a handful of crucial dimensions d.

Value

(Invisibly) A list containing:

`grid_points` A numeric vector of the `nPoints_br` grid values in $[0, 1]$ used for the outer loop.

`br_dag_given_ast` A numeric vector of the same length, giving the dag best-response $\pi_{\text{dag},d}$ at each grid point for $\pi_{\text{ast},d}$.

`br_ast_given_dag` A numeric vector with the ast best-response for each grid point $\pi_{\text{dag},d}$.

See Also

strategize for obtaining the result object res in adversarial mode. See also cv_strategize.

Examples

```
## Not run:
# =====================================================
# Visualize best-response curves in adversarial mode
# =====================================================
# First, fit an adversarial strategize model
set.seed(42)
n <- 400

# Generate data with party structure
W <- data.frame(
  Gender = sample(c("Male", "Female"), n, replace = TRUE),
  Age = sample(c("Young", "Middle", "Old"), n, replace = TRUE)
)

# Party affiliations for respondents and candidates
respondent_party <- sample(c("Dem", "Rep"), n/2, replace = TRUE)
candidate_party <- rep(c("Dem", "Rep"), n/2)

Y <- rbinom(n, 1, 0.5)  # Simplified outcome

# Fit adversarial model
adv_result <- strategize(
  Y = Y,
  W = W,
  lambda = 0.1,
  adversarial = TRUE,
  competing_group_variable_respondent = rep(respondent_party, each = 2),
  competing_group_variable_candidate = candidate_party,
  nSGD = 100
)

# Plot best-response curves for Gender dimension (d_ = 1)
# Shows how each party's optimal Gender distribution responds
# to changes in the other party's Gender distribution
plot_best_response_curves(
  res = adv_result,
  d_ = 1,  # Gender is first factor
  nPoints_br = 50,
  title = "Gender: Best-Response Curves",
  col_ast = "blue",   # Democrats
  col_dag = "red"     # Republicans
)

# Intersection point indicates Nash equilibrium for this dimension

## End(Not run)
```

`plot_convergence` *Plot Convergence Diagnostics for Strategize Results*

Description

Visualizes the optimization trajectory from gradient descent, showing gradient magnitudes, loss values, and learning rate adaptation over iterations.

Usage

```
plot_convergence(
  result,
  metrics = c("gradient", "loss"),
  log_scale = TRUE,
  use_ggplot = FALSE
)
```

Arguments

`result` Output from `strategize` with `adversarial = TRUE`

`metrics` Character vector specifying which metrics to plot. Options are:

- `"gradient"`: Gradient magnitude (L2 norm) over iterations
- `"loss"`: Objective function value over iterations
- `"lr"`: Learning rate adaptation over iterations

Default is `c("gradient", "loss")`.

`log_scale` Logical. Whether to use log scale for gradient magnitudes. Default is `TRUE`.

`use_ggplot` Logical. If `TRUE` and ggplot2 is available, uses ggplot2 for visualization. Otherwise uses base R graphics. Default is `FALSE`.

Details

This function provides diagnostic plots to assess whether the gradient descent optimization has converged to a Nash equilibrium. Key indicators of convergence:

- Gradient magnitudes should decrease toward zero for both players
- Loss values should stabilize (may oscillate slightly in adversarial settings)
- Learning rates should adapt appropriately (decrease as gradients shrink)

In adversarial (two-player) mode, both AST and DAG players' metrics are shown. In non-adversarial mode, only AST metrics are displayed.

Value

If `use_ggplot = TRUE` and ggplot2 is available, returns a ggplot object. Otherwise, plots are created as side effects and the function returns `invisible(NULL)`.

Examples

```
## Not run:
# Run adversarial strategize
result <- strategize(Y = y, W = w, adversarial = TRUE, nSGD = 500)

# Plot convergence diagnostics
plot_convergence(result)

# Plot only gradient magnitudes with log scale
plot_convergence(result, metrics = "gradient", log_scale = TRUE)

## End(Not run)
```

---

plot_quadrant_breakdown *Plot Four-Quadrant Contribution Breakdown*

Description

Decomposes the equilibrium vote share Q* into contributions from four distinct election scenarios, providing insight into which primary-to-general election pathways drive the Nash equilibrium outcome.

Usage

```
plot_quadrant_breakdown(
  result,
  type = c("bar", "pie"),
  nMonte = 500,
  verbose = TRUE
)
```

Arguments

`result`	Output from `strategize` with `adversarial = TRUE`
`type`	Character string specifying the plot type: • `"bar"`: Stacked bar chart showing contributions • `"pie"`: Pie chart showing proportions Default is `"bar"`.
`nMonte`	Integer. Number of Monte Carlo samples for estimation. Default is 500.
`verbose`	Logical. Whether to print progress messages. Default is `TRUE`.

Details

In adversarial mode, two parties simultaneously optimize their candidate attribute distributions. Each party's "entrant" candidate (drawn from the optimized distribution) competes in a primary against a "field" candidate (drawn from the baseline distribution). The general election outcome depends on which candidates win their respective primaries, creating four possible scenarios. This function visualizes the relative importance of each scenario to the overall equilibrium.

The four scenarios (quadrants) represent different primary election outcomes:

E1: Both Entrants Both parties' "entrant" candidates (sampled from optimized distributions) win their primaries

E2: A Entrant, B Field Party A's entrant wins, Party B's field candidate (sampled from baseline) wins

E3: A Field, B Entrant Party A's field wins, Party B's entrant wins

E4: Both Field Both parties' field candidates win their primaries

The weight of each scenario depends on the primary election probabilities (kappa values), which in turn depend on the voter model.

Value

A list containing:

weights Named vector of four-quadrant weights (sum to 1)

contributions Named vector of contributions to Q* from each quadrant

Q_star Total equilibrium vote share

plot Base R plot (invisible return)

Interpretation

- A dominant E1 contribution suggests entrant vs. entrant matchups are most important for the equilibrium
- Balanced contributions indicate a robust equilibrium across scenarios
- Large E4 contribution suggests field candidates often win primaries

Examples

```
## Not run:
# Run adversarial strategize
result <- strategize(Y = y, W = w, adversarial = TRUE, nSGD = 500)

# Plot quadrant breakdown
breakdown <- plot_quadrant_breakdown(result)
print(breakdown$weights)
print(breakdown$contributions)

## End(Not run)
```

`predict.strategic_predictor`

Predict from a fitted strategic predictor

Description

Predict from a fitted strategic predictor

Usage

```
## S3 method for class 'strategic_predictor'
predict(
  object,
  newdata,
  type = c("response", "link"),
  interval = c("none", "ci", "draws"),
  level = 0.95,
  n_draws = 0L,
  seed = NULL,
  factor_schema = NULL,
  text_embedding_fn = NULL,
  ...
)
```

Arguments

`object` A fitted `strategic_predictor`.

`newdata` New data. For `mode="single"`, a data.frame/matrix of factor columns. For `mode="pairwise"`, either:

- a data.frame containing factor columns plus `pair_id` (and optionally `profile_order`), or
- a list with elements `W`, `pair_id`, and optional `profile_order`.

Neural foundation predictors also accept optional `experiment_country` and `experiment_year` as scalar or row-aligned vectors; in pairwise mode row-aligned values are taken from the left/profile-level prediction row.

`type` `"response"` (probability) or `"link"` (logit / linear predictor).

`interval` `"none"` (default), `"ci"`, or `"draws"`.

`level` Credible interval level for draws.

`n_draws` Number of posterior draws when `interval!="none"`.

`seed` Optional seed for draws.

`factor_schema` Optional explicit prediction-time factor schema for fused neural predictors. Supply a list with `names_list` or `p_list`, and optionally precomputed factor/level text or structural matrices.

`text_embedding_fn` Optional text embedding function used with `factor_schema` and prediction-time `experiment_description`.

`...` Unused.

---

prediction-api *Prediction-Only Outcome Model API*

Description

Fit the package's outcome model (GLM or neural) without running the stochastic intervention optimization in `strategize`. Returns a fitted `strategic_predictor` object with a `predict` method.

predict_pair *Predict on wide-format pairwise data*

Description

Predict on wide-format pairwise data

Usage

```
predict_pair(
  fit,
  W_left,
  W_right,
  type = c("response", "link"),
  interval = c("none", "ci", "draws"),
  level = 0.95,
  n_draws = 0L,
  seed = NULL,
  ...
)
```

Arguments

`fit`	A fitted stage-free `strategic_predictor` with `mode="pairwise"`. Stage-aware pairwise predictors require long-format `predict()` input because candidate and respondent group metadata are row-aligned.
`W_left`	Data frame/matrix of left profiles.
`W_right`	Data frame/matrix of right profiles.
`type`	`"response"` or `"link"`.
`interval`	`"none"`, `"ci"`, or `"draws"`.
`level`	Credible interval level for draws.
`n_draws`	Number of posterior draws when `interval!="none"`.
`seed`	Optional seed for draws.
`...`	Unused.

Value

Predictions for each row-pair.

`print.hessian_analysis` *Print method for hessian_analysis objects*

Description

Print method for hessian_analysis objects

Usage

```
## S3 method for class 'hessian_analysis'
print(x, ...)
```

Arguments

`x`	A hessian_analysis object from `check_hessian_geometry`
`...`	Additional arguments (ignored)

`print.strategize_result` *Print Method for strategize Results*

Description

Print Method for strategize Results

Usage

```
## S3 method for class 'strategize_result'
print(x, digits = 3, ...)
```

Arguments

`x`	A strategize result object
`digits`	Number of digits to display
`...`	Additional arguments (ignored)

Value

Invisibly returns the input object

save_conjoint_foundation_bundle
Conjoint foundation bundle writing moved to preference.fm

Description

Conjoint foundation bundle writing moved to preference.fm

Usage

```
save_conjoint_foundation_bundle(
  file,
  foundation_model,
  overwrite = FALSE,
  compress = TRUE
)
```

Arguments

`file`	Path to save the bundle.
`foundation_model`	A fitted `conjoint_foundation_model`.
`overwrite`	Logical; overwrite any existing file.
`compress`	Retained for API compatibility.

Details

This exported name is retained for migration guidance. New foundation artifacts are checkpoint directories written by `preference.fm::save_conjoint_foundation_bundle()`.

Value

The bundle path (invisibly).

See Also

`load_conjoint_foundation_bundle()`

Examples

```
## Not run:
preference.fm::save_conjoint_foundation_bundle(
  tempfile(),
  foundation_model = foundation_fit,
  overwrite = TRUE
)

## End(Not run)
```

save_neural_outcome_bundle
Save a portable neural outcome bundle

Description

Save a portable neural outcome bundle

Usage

```
save_neural_outcome_bundle(
  file,
  theta_mean,
  theta_var = NULL,
  neural_model_info,
  names_list = NULL,
  p_list = NULL,
  factor_levels = NULL,
  mode = c("auto", "pairwise", "single"),
  fit_metrics = NULL,
  conda_env = "strategize_env",
  conda_env_required = TRUE,
  overwrite = FALSE,
  compress = TRUE,
  metadata = NULL
)
```

Arguments

`file`	Path to save the bundle (typically ending in `.rds`).
`theta_mean`	Numeric vector of posterior means for neural parameters.
`theta_var`	Optional numeric vector of posterior variances.
`neural_model_info`	Neural model metadata (can include reticulate objects).
`names_list`	Optional list of factor level names (see `cs_prepare_W_encoding`).
`p_list`	Optional `p_list` to derive factor level names when `names_list` is missing.
`factor_levels`	Optional integer vector of factor levels (derived from `names_list` by default).
`mode`	`"auto"`, `"pairwise"`, or `"single"`.
`fit_metrics`	Optional fit metrics to include.
`conda_env`	Conda env name for neural backend (stored in metadata).
`conda_env_required`	Require conda env to exist (stored in metadata).
`overwrite`	Logical; overwrite existing file.
`compress`	Compression passed to `saveRDS()`.
`metadata`	Optional list of extra metadata to include.

Value

The bundle path (invisibly).

save_strategic_predictor

Save a strategic predictor to disk

Description

Save a strategic predictor to disk

Usage

```
save_strategic_predictor(
  fit,
  file,
  overwrite = FALSE,
  compress = TRUE,
  include_metrics = TRUE
)
```

Arguments

`fit`	A fitted `strategic_predictor`.
`file`	Path to save the cache (typically ending in `.rds`).
`overwrite`	Logical; overwrite an existing file.
`compress`	Compression passed to `saveRDS()`.
`include_metrics`	Logical; include out-of-sample fit metrics when present.

Value

The cache path (invisibly).

strategic_prediction

Fit a prediction-only outcome model

Description

Fit a prediction-only outcome model

Usage

```
strategic_prediction(
  Y,
  W,
  X = NULL,
  ...,
  model = c("glm", "neural"),
  mode = c("auto", "pairwise", "single"),
  pair_id = NULL,
```

```
  profile_order = NULL,
  varcov_cluster_variable = NULL,
  conda_env = "strategize_env",
  conda_env_required = TRUE,
  neural_mcmc_control = NULL,
  use_regularization = TRUE,
  nFolds_glm = 3L,
  cache_path = NULL,
  cache_overwrite = FALSE,
  cache_compress = TRUE
)
```

Arguments

`Y`	Binary outcome in `0/1`.
`W`	Factor matrix/data.frame (one column per conjoint factor).
`X`	Optional covariate matrix/data.frame (neural backend).
`...`	Reserved for future extensions.
`model`	`"glm"` or `"neural"`.
`mode`	`"auto"`, `"pairwise"`, or `"single"`.
`pair_id`	Optional pair identifier (required for pairwise).
`profile_order`	Optional within-pair ordering (1/2).
`varcov_cluster_variable`	Optional cluster IDs for robust GLM vcov.
`conda_env`	Conda env name for neural backend.
`conda_env_required`	Require conda env to exist (neural backend).
`neural_mcmc_control`	Optional list passed to neural backend.
`use_regularization`	Logical; run glinternet screening for GLM.
`nFolds_glm`	Number of folds for glinternet CV (GLM).
`cache_path`	Optional path to a cached predictor (.rds). If it exists and `cache_overwrite` is `FALSE`, the cached model is loaded instead of refitting. When a new model is fit, it is saved to this path. For neural fits, `cache_path` also enables restartable SVI checkpoints under `paste0(cache_path, ".inprogress")`; successful final saves remove that in-progress directory.
`cache_overwrite`	Logical; refit and overwrite any existing cache at `cache_path`.
`cache_compress`	Compression setting passed to `saveRDS()`.

Value

An object of class `strategic_predictor`.

`strategize` *Estimate Optimal (or Adversarial) Stochastic Interventions for Conjoint Experiments*

Description

`strategize` implements the core methods described in the accompanying paper for learning an optimal or adversarial probability distribution over conjoint factor levels. It is specifically designed for forced-choice conjoint settings (e.g., candidate-choice experiments) and can accommodate scenarios in which a single agent optimizes its strategy in isolation, or in which two (potentially adversarial) agents simultaneously optimize against each other.

This function can be used to find the *optimal stochastic intervention* for maximizing an outcome of interest (e.g., vote choice, rating, or utility), possibly subject to a penalty that keeps the learned distribution close to the original design distribution. It can also incorporate institutional rules (e.g., primaries, multiple stages of choice) by specifying additional arguments. Estimation can be done under standard generalized linear modeling assumptions or more advanced approaches. The function returns estimates of the learned distribution and the associated performance quantity ($Q(\boldsymbol{\pi}^*)$) along with optional inference based on the (asymptotic) delta method.

Usage

```
strategize(
  Y,
  W,
  X = NULL,
  lambda,
  varcov_cluster_variable = NULL,
  competing_group_variable_respondent = NULL,
  competing_group_variable_respondent_proportions = NULL,
  competing_group_variable_candidate = NULL,
  competing_group_competition_variable_candidate = NULL,
  pair_id = NULL,
  respondent_id = NULL,
  respondent_task_id = NULL,
  profile_order = NULL,
  p_list = NULL,
  slate_list = NULL,
  K = 1,
  nSGD = 100,
  diff = FALSE,
  adversarial = FALSE,
  adversarial_model_strategy = "four",
  include_stage_interactions = NULL,
  partial_pooling = NULL,
  partial_pooling_strength = 50,
  use_regularization = TRUE,
  force_gaussian = FALSE,
  force_reinforce = FALSE,
  a_init_sd = 0.001,
  outcome_model_type = "glm",
```

```
  neural_mcmc_control = NULL,
  penalty_type = "KL",
  compute_se = FALSE,
  se_method = c("full", "implicit"),
  conda_env = "strategize_env",
  conda_env_required = FALSE,
  conf_level = 0.9,
  nFolds_glm = 3L,
  folds = NULL,
  crossfit_q = FALSE,
  crossfit_q_control = NULL,
  nMonte_adversarial = 5L,
  primary_pushforward = "mc",
  primary_strength = 1,
  primary_n_entrants = 1L,
  primary_n_field = 1L,
  nMonte_Qglm = 100L,
  learning_rate_max = 0.001,
  temperature = NULL,
  save_outcome_model = FALSE,
  presaved_outcome_model = FALSE,
  outcome_model_key = NULL,
  use_optax = FALSE,
  optim_type = "gd",
  optimism = "extragrad",
  optimism_coef = 1,
  rain_lambda = 1,
  rain_gamma = 0.01,
  rain_L = NULL,
  rain_eta = 0.001,
  rain_variant = "alg10_staged",
  rain_output = "last",
  compute_hessian = TRUE,
  hessian_max_dim = 50L
)
```

Arguments

Y — A numeric or binary vector of observed outcomes, typically in {0,1} for forced-choice conjoint tasks, indicating whether the profile was selected. For instance, Y = 1 if candidate A was chosen over candidate B, and Y = 0 otherwise. The length must match the number of rows in W.

W — A matrix or data frame representing the assigned levels of each factor in a conjoint design (one column per factor). Each row corresponds to a single profile. For forced-choice tasks, a given respondent may have contributed multiple rows if you reshape pairwise choices into long format. If the experiment used multiple factors D, with each factor having L_d levels, W should capture all factor assignments accordingly.

X — An optional matrix or data frame of additional covariates, often respondent-level features (e.g., respondent demographics). If K > 1, X may be used internally to fit multi-cluster or multi-component outcome models, or to allow cluster-specific effect estimation for more granular insights. Defaults to NULL.

`lambda` A numeric scalar or vector giving the regularization penalty (e.g., in Kullback-Leibler or L2 sense) used to shrink the learned probability distribution(s) of factor levels toward a baseline distribution `p_list`. Typically set via either domain knowledge or cross-validation.

`varcov_cluster_variable`
: An optional vector of cluster identifiers (e.g., respondent IDs) used to form a robust variance-covariance estimate of the outcome model. If `NULL`, the usual IID assumption is made. Defaults to `NULL`.

`competing_group_variable_respondent`
: Optional variable marking competition group membership of respondents. Particularly relevant in adversarial settings (`adversarial = TRUE`) or multi-stage electoral settings, e.g., capturing the party of each respondent. Defaults to `NULL`.

`competing_group_variable_respondent_proportions`
: Optional numeric vector specifying the population proportions of each competing group. If `NULL`, proportions are estimated from the data. Useful when the sample proportions differ from the target population proportions. Defaults to `NULL`.

`competing_group_variable_candidate`
: Optional variable marking competition group membership of candidate profiles. Defaults to `NULL`.

`competing_group_competition_variable_candidate`
: Optional variable indicating whether a candidate profile belongs to the competing group in adversarial settings. Defaults to `NULL`.

`pair_id` A factor or numeric vector identifying the forced-choice pair. If each row of `W` is a single profile, `pair_id` groups the rows belonging to the same choice set. Defaults to `NULL`.

`respondent_id`, `respondent_task_id`
: Another set of optional identifiers. `respondent_id` marks each respondent across tasks, while `respondent_task_id` can define unique IDs for repeated measurements from the same respondent across multiple tasks. Useful for advanced clustering or robust SEs. Defaults to `NULL`.

`profile_order` If each forced-choice is shown with different ordering (e.g., `Candidate A` vs. `Candidate B`), `profile_order` can label each row accordingly. Helpful for ensuring consistent labeling of reference vs. opposing profiles. Defaults to `NULL`.

`p_list` An optional list describing the baseline probability distribution over factor levels in `W`. Typically derived from the initial design distribution or uniform assignment distribution. If `NULL`, the function may assume uniform or attempt to estimate the distribution from `W`.

`slate_list` An optional list (or lists) providing custom slates of candidate features (and their associated probabilities). Used in more advanced or adversarial setups where certain combinations must be included or excluded. If `NULL`, no special constraints beyond the usual factor-level distributions are applied.

`K` Integer specifying the number of latent clusters for multi-component outcome models. If `K = 1`, no latent clustering is done. Defaults to `1`.

`nSGD` Integer specifying the number of stochastic gradient descent (or gradient-based) iterations to use when learning the optimal distributions. Defaults to `100`.

`diff` Logical indicating whether the outcome `Y` represents a first-difference or difference-based metric. In forced-choice contexts, typically `diff = FALSE`. Defaults to `FALSE`.

adversarial
: Logical controlling whether to enable the max-min adversarial scenario. When `TRUE`, the function searches for a pair of distributions (one for each competing party or group) such that each party's distribution is optimal given the other party's distribution. Defaults to `FALSE`.

adversarial_model_strategy
: Character string indicating whether to estimate `"four"` outcome models (primary + general for each group), `"two"` outcome models (one per group reused for both primary and general), or `"neural"` (Bayesian Transformer models with party tokens; defaults to a single pooled model across groups and stages. Set `neural_mcmc_control$n_bayesian_models = 2` to fit separate AST/DAG models). Defaults to `"four"`.

include_stage_interactions
: Logical indicating whether to include stage (primary vs general) indicator and stage-by-factor interactions in the outcome model. When `NULL` (default), automatically set to `TRUE` when `adversarial_model_strategy = "two"` and `FALSE` otherwise. Including stage interactions allows a single pooled model to learn different response patterns for primary vs general election scenarios, which helps prevent pattern-matching equilibria where both parties converge to identical strategies.

partial_pooling
: Logical indicating whether to partially pool (shrink) group-specific outcome model coefficients toward a shared average when `adversarial_model_strategy = "two"`. When `NULL` (default), automatically set to `TRUE` for the two-strategy adversarial case and `FALSE` otherwise.

partial_pooling_strength
: Numeric scalar controlling the amount of shrinkage used for partial pooling in the two-strategy adversarial case. Interpreted as a pseudo-sample size: larger values increase pooling, smaller values preserve group differentiation. Defaults to `50`.

use_regularization
: Logical indicating whether to regularize the outcome model (in addition to any penalty `lambda` on the distribution shift). This can help avoid overfitting in high-dimensional designs. Defaults to `TRUE`.

force_gaussian
: Logical indicating whether to force a Gaussian-based outcome modeling approach, even if `Y` is binary or forced-choice. If `FALSE`, the function attempts to choose a more appropriate link (e.g., `"binomial"`). Defaults to `FALSE`.

force_reinforce
: Logical indicating whether to force REINFORCE-based optimization for the neural average-case objective when exact small-support enumeration is available. This only affects the optimization objective path; reported `Q` values continue to use the existing exact report-time evaluation when available. Defaults to `FALSE`.

a_init_sd
: Numeric scalar specifying the standard deviation for random initialization of unconstrained parameters used in the gradient-based search over factor-level probabilities. Defaults to `0.001`.

outcome_model_type
: Character string specifying the outcome model to use. Currently supports `"glm"` for generalized linear models or `"neural"` for a neural-network approximation. Defaults to `"glm"`.

neural_mcmc_control
: Optional list overriding default MCMC settings used when `outcome_model_type = "neural"`. Named entries override the defaults in `two_step_model_outcome_neural.R`.

Set neural_mcmc_control$uncertainty_scope = "output" to compute delta-method uncertainty using only the output-layer parameters (default is "all"). In adversarial neural mode, set neural_mcmc_control$n_bayesian_models = 2 to fit separate AST/DAG models (default is 1 for a single differential model). Use neural_mcmc_control$ModelDims and neural_mcmc_control$ModelDepth to override the Transformer hidden width and depth. Pairwise neural fits default to neural_mcmc_control$cross_candidate_encoder = "term" when unspecified, except rank-positive low-rank respondent-candidate interaction defaults to "none" to avoid duplicating pairwise interaction capacity. Set neural_mcmc_control$cross_candi = "term" (or TRUE) to include the opponent-dependent cross-candidate term in pairwise mode, set neural_mcmc_control$cross_candidate_encoder = "attn" to add a lightweight cross-attention step. When combined with neural_mcmc_control$resi = "full_attn", that step consumes the depth-attended candidate-token readout. Set neural_mcmc_control$cross_candidate_encoder = "full" to enable a full cross-encoder that jointly encodes both candidates. Use "none" (or FALSE) to disable. Set neural_mcmc_control$qk_norm = TRUE (default) to apply RMSNorm to projected queries and keys before forming attention logits. Set neural_mcmc_control$residual_mode = "full_attn" to replace the default additive/ReZero residual path with full depth-wise attention over all prior layer outputs plus a final depth-attentive readout; use "standard" (default) to keep the existing residual formulation. Rank-positive low-rank Bernoulli pairwise logits use RMS logit normalization by default; set neural_mcmc_control$low_rank_logit_normal = "none" to disable it. Explicit low_rank_logit_transform = "softclip" remains supported for compatibility. For variational inference (subsample_method = "batch_vi"), set neural_mcmc_control$optimizer to "muon" (default when optax.contrib.muon is available), "adam" (numpyro.optim), "adamw" (AdamW), or "adabelief" (optax). Muon is only applied to matrix-valued model-weight locations when the guide preserves matrix structure; with vi_guide = "auto_diagonal", it falls back to AdamW/Adam. Learning-rate decay is controlled by neural_mcmc_control$svi_ste (integer steps, or "optimal" for a scaling-law heuristic based on model/data size; for minibatched VI this also scales with batch_size) and neural_mcmc_control$svi_lr_sche (default "warmup_cosine"), with optional svi_lr_warmup_frac and svi_lr_end_factor. Validation-based early stopping is enabled by default for SVI-backed neural fits; set neural_mcmc_control$early_stopping = FALSE to force the full svi_steps budget. Use neural_mcmc_control$early_stopping_n_checks (default 10) to request an approximate number of validation checks during SVI early stopping; the cadence is derived as ceiling(svi_steps / early_stopping_n_checks). Use neural_mcmc_control$early_stopping_patience (default 3) to control how many consecutive non-improving validation checks are tolerated before stopping. For compact streaming SVI, early_stopping = TRUE enables validation best-checkpoint selection but still runs the full svi_steps budget. Set neural_mcmc_control$early_stopping_validation_frac (default 0.05) to retain approximately that fraction of evaluable observations in the held-out validation split, and optionally neural_mcmc_control$early_stopping_validation_max_n (default 2048) to cap the retained validation size after fraction-based sizing. Set early_stopping_validation_max_n = NULL to disable that cap. Checkpoint gradient diagnostics are enabled by default for SVI fits and are evaluated at the early-stopping checkpoint cadence; set neural_mcmc_control$gradient_diagnostics = FALSE to disable them. Advanced restart checkpoints can be enabled with neural_mcmc_control$checkpoint_path; rerunning with the same data and controls resumes from latest.rds and promotes the best validation checkpoint when present. checkpoint_resume defaults to TRUE when a checkpoint path is present, checkpoint_n_checks defaults to 10 for non-early-stopping checkpoint cadence, and checkpoint_compress defaults to FALSE.

penalty_type
A character string specifying the type of penalty (e.g., "KL", "L2", or "LogMaxProb") used in the objective function for shifting the factor-level probabilities away from the baseline p_list. Defaults to "KL".

compute_se
Logical indicating whether standard errors should be computed for the final estimates (via the delta method or related expansions). Defaults to FALSE.

se_method
Character string specifying the SE computation method when compute_se = TRUE. "full" differentiates through the full optimization trace (default). "implicit" uses implicit differentiation at the solution (adversarial equilibrium or non-adversarial optimum).

conda_env
A character string naming a Python conda environment that includes **jax**, **optax**, and other dependencies used by the optimization workflow. If not NULL, the function attempts to activate that environment. Defaults to "strategize_env".

conda_env_required
Logical; if TRUE, raises an error if the environment given by conda_env cannot be activated. Otherwise, the function attempts to proceed with any available installation. Defaults to FALSE.

conf_level
Numeric in $(0, 1)$, specifying the confidence level for intervals or credible bounds. Defaults to 0.90.

nFolds_glm
Integer specifying the number of folds (default 3L) for internal cross-validation used in certain outcome model or regularization steps. Defaults to 3L.

folds
An optional user-supplied partitioning or CV scheme, overriding nFolds_glm. Defaults to NULL.

crossfit_q
Logical. If TRUE, compute a cross-fitted policy-value diagnostic for supported pairwise forced-choice binomial GLM runs. This currently includes non-adversarial average-case runs and adversarial adversarial_model_strategy = "four" runs with exactly two respondent and candidate groups. The returned fields Q_crossfit, Q_reference_crossfit, and Q_gain_crossfit summarize out-of-fold value for the learned policy relative to the assignment-policy baseline.

crossfit_q_control
Optional list controlling cross-fitted Q evaluation. Supported entries include folds, seed, split_by, estimators, headline, weight_clip, n_policy_draws, chunk_size, return_fold_results, and perspective_group. For adversarial runs, perspective_group selects the candidate group whose win probability is reported; if omitted, the first sorted group label is used. The adversarial estimator assumes independent group-specific profile assignment under p_list. The default headline estimator is doubly robust ("dr"), with IPS, SNIPS, and model-only diagnostics returned in Q_crossfit_info.

nMonte_adversarial
Integer specifying the number of Monte Carlo samples used in adversarial or max-min steps, e.g., sampling from the opposing candidate's distribution to approximate expected payoffs. Defaults to 5L.

primary_pushforward
Character string controlling the primary-stage push-forward estimator. Use "mc" (default) for Monte Carlo sampling with per-draw primary winners, or "linearized" for the faster averaged-weight approximation, or "multi" for multi-candidate primaries.

primary_strength
: Numeric scalar controlling primary decisiveness. Values > 1 make primary outcomes more deterministic; values in (0, 1) make primaries more noisy. Defaults to 1.0 (neutral scaling).

primary_n_entrants
: Integer number of entrant candidates sampled per party in multi-candidate primaries (`primary_pushforward = "multi"`). Defaults to 1.

primary_n_field
: Integer number of field candidates sampled per party in multi-candidate primaries (`primary_pushforward = "multi"`). Defaults to 1.

nMonte_Qglm
: Integer specifying the number of Monte Carlo samples for evaluating or approximating the quantity of interest under certain outcomes or distributions. Defaults to `100L`.

learning_rate_max
: Base learning rate for gradient-based optimizers. Defaults to `0.001`.

temperature
: Numeric temperature parameter used in Gumbel-Softmax sampling to smooth the exploration of the probability simplex. Smaller values yield distributions closer to the argmax. Defaults to `0.5`.

save_outcome_model
: Logical indicating whether to save the fitted outcome model to disk for reuse. Useful for large models or repeated runs. Defaults to `FALSE`.

presaved_outcome_model
: Logical indicating whether to use a previously saved outcome model instead of re-fitting. Defaults to `FALSE`.

outcome_model_key
: Optional character string to append to saved outcome model filenames. Useful for distinguishing between different model configurations or experimental runs. When provided, files are saved as `main_{group}_{round}_{key}.csv`. Defaults to `NULL`.

use_optax
: Logical indicating whether to use the `optax` library for gradient-based optimization in JAX (`TRUE`) or a built-in method (`FALSE`). Defaults to `FALSE`.

optim_type
: A character string for choosing which optimizer or approach is used internally (e.g., `"gd"` for gradient descent). Defaults to `"gd"`.

optimism
: Character string controlling optimistic / extra-gradient updates for the gradient optimizer. Options: `"extragrad"` (default; classic Korpelevich extra-gradient), `"smp"` (stochastic mirror-prox: extra-gradient with weighted averaging of lookahead points), `"ogda"` (optimistic gradient), `"rain"` (RAIN: Recursive Anchored Iteration with anchored extra-gradient and increasing quadratic anchor penalties), or `"none"` (standard updates). Only supported when `use_optax = FALSE`.

optimism_coef
: Numeric scalar controlling the magnitude of optimism adjustments. For `"ogda"`, this scales the optimistic correction term. Ignored when `optimism = "rain"`.

rain_lambda
: Numeric scalar giving the base RAIN regularization scale λ. The staged algorithm uses $\lambda_0 = \gamma\lambda$ and grows by $(1+\gamma)$ each stage. Only used when `optimism = "rain"`.

rain_gamma
: Non-negative numeric scalar for the RAIN anchor-growth parameter γ. Larger values grow anchor weights faster. If not supplied, the default is auto-scaled downward when `nSGD` exceeds 100 to keep total anchor growth roughly constant. Only used when `optimism = "rain"`.

`rain_L` Optional numeric scalar for the Lipschitz constant L used by RAIN. When supplied and `rain_eta` is missing, `rain_eta` defaults to $1/(8L)$ and stage lengths follow $T_s = 16L/\lambda_s$. If `NULL`, L is estimated from `rain_eta`. Only used when `optimism = "rain"`.

`rain_eta` Optional numeric scalar step size η for RAIN. Defaults to `0.001` and is auto-scaled downward when `nSGD` exceeds 100 if not supplied. If `rain_L` is supplied and `rain_eta` is missing, defaults to $1/(8L)$. Only used when `optimism = "rain"`.

`rain_variant` Character string specifying the RAIN variant. Options: `"alg10_staged"` (merged Algorithm 10 with recursive stage anchors; default) or `"alg9_single_loop"` (single-loop variant; not yet implemented). Only used when `optimism = "rain"`.

`rain_output` Character string controlling the stage output for RAIN. `"uniform_half"` samples uniformly from half-iterates within the stage (most faithful); `"last"` returns the last iterate (default). Only used when `optimism = "rain"`.

`compute_hessian`
Logical. Whether to compute Hessian functions for equilibrium geometry analysis in adversarial mode. When `TRUE` (default), Hessian functions are JIT-compiled to enable `check_hessian_geometry` analysis. Set to `FALSE` to skip Hessian computation for faster execution.

`hessian_max_dim`
Integer. Maximum number of parameters per player before automatically skipping Hessian computation. Defaults to `50L`. For problems with more parameters, Hessian computation is skipped to avoid memory/time overhead. The result will have `hessian_skipped_reason = "high_dimension"` in this case.

Details

Modeling the outcome: Internally, `strategize` may fit a generalized linear model or a more flexible approach (such as multi-cluster factorization) to learn the mapping from factor-level assignments `W` (and optional covariates `X`) onto outcomes `Y`. Once these outcome coefficients are estimated, the function uses gradient-based or closed-form solutions to find the *optimal stochastic intervention(s)*, i.e., new factor-level probability distributions that maximize an expected outcome (or solve the max-min adversarial problem).

Adversarial or strategic design: When `adversarial = TRUE`, the function attempts to solve a zero-sum game in which one agent (say, "A") chooses its distribution to maximize vote share, while the other ("B") simultaneously chooses its distribution to minimize "A"'s vote share. In many settings, `competing_group_variable_respondent` and related arguments help define which respondents belong to the "A" or "B" sub-electorate (e.g., a primary). The final solution is a mixed-strategy Nash equilibrium, if it exists, for the forced-choice environment. This can be used to compare or interpret real-world candidate positioning in multi-stage elections.

Regularization: The argument `lambda` penalizes how far the learned distribution strays from the baseline distribution `p_list`. This helps avoid overfitting in high-dimensional designs. Different penalty types can be selected via `penalty_type`.

Implementation details: Under the hood, this function may rely on **jax** for automatic differentiation. By default, it uses an internal gradient-based approach. If `use_optax = TRUE`, the `optax` library is used for optimization. The function can automatically detect or load a **conda** environment if specified, though advanced users can pass `conda_env_required = TRUE` to enforce that environment activation is mandatory.

Value

A named `list` containing:

`pi_star_point` An estimate of the (possibly multi-cluster or adversarial) optimal distribution(s) over the factor levels.

Structure depends on parameters:

- If `adversarial = TRUE` and `K = 1`, returns a pair of distributions (e.g., maximin solutions).
- If `K > 1`, returns a list where each element corresponds to a cluster-optimal distribution.
- Otherwise, returns a single distribution.

`pi_star_se` Standard errors for entries in `pi_star_point`. Mirrors the structure of `pi_star_point` (e.g., a pair of SEs if `adversarial = TRUE` and `K = 1`). Only present if `compute_se = TRUE`.

`Q_point` Primary point estimate(s) of the optimized outcome (e.g., utility or vote share). Matches the structure of `pi_star_point`.

`Q_se` Primary standard errors for `Q_point`. Only present if `compute_se = TRUE`.

`Q_point_mEst`, `Q_se_mEst` Backward-compatible aliases for `Q_point` and `Q_se`.

`pi_star_lb`, `pi_star_ub` Confidence bounds for `pi_star_point` (if `compute_se = TRUE` and a confidence level is provided).

`outcome_model_view` Interpretable summaries of the fitted outcome models (by player and stage). Includes main-effect tables and top interactions for AST/DAG primary/general submodels when available.

`CVInfo` Cross-validation performance data (if applicable). Typically a `data.frame` or list.

`Q_crossfit`, `Q_reference_crossfit`, `Q_gain_crossfit` Optional cross-fitted policy-value diagnostics returned when `crossfit_q = TRUE`.

`Q_crossfit_info` Optional fold-level and estimator-level diagnostics for cross-fitted Q evaluation, including IPS/SNIPS/model/DR summaries and importance-weight effective sample size. For adversarial runs this also records the perspective/opponent groups, target and reference summaries, and the independent-product assignment assumption used for profile weights.

`estimationType` String indicating the approach used by the active estimator, `"TwoStep"`.

`...` Additional internal details (e.g., fitted models, optimization logs).

See Also

`cv_strategize` for cross-validation across candidate values of `lambda`.

Examples

```
# ============================================
# Example 1: Basic single-agent optimization
# ============================================
# Generate synthetic conjoint data
set.seed(42)
n <- 400  # Number of profiles (200 pairs)

# Factor matrix: candidate attributes
W <- data.frame(
  Gender = sample(c("Male", "Female"), n, replace = TRUE),
  Age = sample(c("Young", "Middle", "Old"), n, replace = TRUE),
  Party = sample(c("Dem", "Rep"), n, replace = TRUE)
)
```

```
# Simulate outcome: Female + Young candidates preferred
latent <- 0.3 * (W$Gender == "Female") +
          0.2 * (W$Age == "Young") -
          0.1 * (W$Age == "Old")
prob <- plogis(latent)

# Paired forced-choice: within each pair, one wins
pair_id <- rep(1:(n/2), each = 2)
Y <- numeric(n)
for (p in unique(pair_id)) {
  idx <- which(pair_id == p)
  winner <- sample(idx, 1, prob = prob[idx])
  Y[idx] <- as.integer(seq_along(idx) == which(idx == winner))
}
profile_order <- rep(1:2, n/2)

# Baseline probabilities (uniform assignment)
p_list <- list(
  Gender = c(Male = 0.5, Female = 0.5),
  Age = c(Young = 1/3, Middle = 1/3, Old = 1/3),
  Party = c(Dem = 0.5, Rep = 0.5)
)

# Run strategize to find optimal distribution
# (requires conda environment with JAX - see build_backend())
result <- strategize(
  Y = Y,
  W = W,
  lambda = 0.1,
  pair_id = pair_id,
  respondent_id = pair_id,
  respondent_task_id = pair_id,
  profile_order = profile_order,
  p_list = p_list,
  diff = TRUE,
  nSGD = 50,
  compute_se = FALSE
)

# View optimal distribution
print(result$pi_star_point)

# View expected outcome under optimal strategy
print(result$Q_point)
```

`strategize.plot` *Plot Estimated Probabilities for Hypothetical Scenarios*

Description

This function creates a grid of base R plots to visualize and compare probabilities (and optionally their confidence intervals) across multiple hypothetical or assignment scenarios. By default, it arranges the plots in a 3xN grid, labeling each row according to the factor or condition being displayed.

Usage

```
strategize.plot(
  pi_star_list = NULL,
  pi_star_se_list = NULL,
  p_list = NULL,
  col.main = "black",
  cex.main = 1.5,
  zStar = 1,
  xlim = NULL,
  ticks_type = "assignmentProbs",
  col_vec = NULL,
  plot_names = TRUE,
  plot_ci = TRUE,
  widths_vec,
  heights_vec,
  main_title = "",
  margins_vec = NULL,
  add = FALSE,
  pch = 20,
  factor_name_transformer = function(x) {
     x
 },
  level_name_transformer = function(x) {
     x
 },
  label_wrap_width = 30,
  label_line_height = 0.6,
  cex.axis = 1.25,
  open_browser = FALSE
)
```

Arguments

`pi_star_list` A list of numeric vectors, each corresponding to a set of hypothetical probabilities to be plotted. These are typically model-based or derived values.

`pi_star_se_list` A list of numeric vectors of the same structure as `pi_star_list`, containing standard errors for each probability. Used to plot confidence intervals.

`p_list` A list of numeric vectors of "assignment" or baseline probabilities to be overlaid as vertical ticks on each plot (depending on `ticks_type`).

`col.main` Character. Color for the main title in each subplot. Default is `"black"`.

`cex.main` Numeric. Character expansion factor for main titles. Default is `1.5`.

`zStar` Numeric. Multiplier for the standard error bars (e.g., 1.96 for approximately 95 percent confidence intervals). Default is `1`.

`xlim` Numeric vector of length 2. The x-axis limits for all subplots. Defaults to `c(0, 1)` if not specified.

`ticks_type` Character. Controls the type of reference ticks added. `"assignmentProbs"`: Vertical ticks drawn at the positions from `p_list` (default). `"zero"`: Vertical ticks drawn at 0. `"none"`: No vertical reference ticks.

col_vec
: Optional character vector of colors (one per set of probabilities in pi_star_list). If NULL, uses sequential indexing for color.

plot_names
: Logical. If TRUE (default), factor/condition labels will be placed along the y-axis.

plot_ci
: Logical. If TRUE (default), error bars will be drawn using pi_star_se_list.

widths_vec, heights_vec
: Currently unused. Reserved for future layout expansions.

main_title
: Character. An overall title for the plot. Default is an empty string.

margins_vec
: Currently unused. Reserved for future layout expansions.

add
: Logical. If FALSE (default), a new plot is created. If TRUE, points/error bars are added to an existing plot space.

pch
: Numeric or character. The plotting symbol. Default is 20.

factor_name_transformer
: Function to transform factor names for display. Should accept and return a character vector. Default is identity function.

level_name_transformer
: Function to transform level names for display. Should accept and return a character vector. Default is identity function.

label_wrap_width
: Numeric. If provided, long level labels are wrapped to this character width using line breaks. Default is 30. Use NULL to disable wrapping.

label_line_height
: Numeric. Additional vertical spacing (in plot units) added per wrapped line beyond the first. Default is 0.6.

cex.axis
: Numeric. Character expansion factor for axis labels. Default is 1.25.

open_browser
: Logical. If TRUE, opens a browser for debugging. Default is FALSE. Intended for development use only.

Details

strategize.plot arranges multiple subplots (3 columns by default) in a grid that depends on the number of elements in p_list. Each subplot will show a factor level or condition on the y-axis, with probabilities along the x-axis. If confidence intervals are provided (pi_star_se_list), horizontal error bars around each probability point will be displayed. Additionally, vertical reference ticks can be added, showing values from p_list or zero depending on ticks_type.

Value

Invisibly returns NULL. This function is primarily called for its side effect: producing a multi-panel base R plot.

Examples

```
# =============================================
# Visualize optimal vs baseline distributions
# =============================================
# This function works without JAX - just needs the result structure

# Create mock strategize result for plotting
# (In practice, use output from strategize())
```

```
pi_star_list <- list(k1 = list(
  Gender = c(Male = 0.35, Female = 0.65),
  Age = c(Young = 0.45, Middle = 0.30, Old = 0.25),
  Party = c(Dem = 0.40, Rep = 0.60)
))

pi_star_se_list <- list(k1 = list(
  Gender = c(Male = 0.04, Female = 0.04),
  Age = c(Young = 0.03, Middle = 0.03, Old = 0.03),
  Party = c(Dem = 0.05, Rep = 0.05)
))

# Baseline (original assignment) probabilities
p_list <- list(
  Gender = c(Male = 0.5, Female = 0.5),
  Age = c(Young = 0.33, Middle = 0.33, Old = 0.34),
  Party = c(Dem = 0.5, Rep = 0.5)
)

# Plot comparing optimal to baseline
strategize.plot(
  pi_star_list = pi_star_list,
  pi_star_se_list = pi_star_se_list,
  p_list = p_list,
  main_title = "Optimal vs Baseline Distribution",
  ticks_type = "assignmentProbs"  # Show baseline as reference ticks
)
```

`strategize_fm_backend` *Backend bridge for preference foundation-model training*

Description

Returns the small set of internal neural/JAX helpers needed by external foundation-model training packages. This is a developer bridge for **preference.fm**; user workflows should call the public strategize APIs.

Usage

```
strategize_fm_backend()
```

Value

A named list of helper functions and the shared JAX environment.

strategize_preset *Get Recommended Parameter Settings*

Description

Returns a list of parameter values suitable for passing to `strategize()`. This simplifies the API by providing sensible defaults for common use cases.

Usage

```
strategize_preset(
  preset = c("standard", "quick_test", "publication", "adversarial")
)
```

Arguments

`preset` One of:

- `"quick_test"` Minimal iterations for testing code (not for inference)
- `"standard"` Reasonable defaults for most analyses (default)
- `"publication"` Higher accuracy for publication-quality results
- `"adversarial"` Settings tuned for adversarial/game-theoretic mode

Details

These presets provide starting points. You should still set data-specific parameters like `Y`, `W`, `p_list`, and potentially `lambda`.

For the "publication" preset, `lambda` is not included because you should use `cv_strategize()` to select it via cross-validation.

Value

Named list of parameters that can be merged with `strategize()` arguments.

Examples

```
# Get standard settings
params <- strategize_preset("standard")
print(params)

# Use with strategize (hypothetically)
# result <- do.call(strategize, c(list(Y = Y, W = W, p_list = p_list), params))
```

summarize_adversarial *Summarize Adversarial Strategize Results*

Description

Prints a comprehensive summary of adversarial equilibrium results, including strategies, equilibrium quality metrics, and four-quadrant breakdown.

Usage

```
summarize_adversarial(
  result,
  validate = TRUE,
  check_hessian = TRUE,
  verbose = TRUE
)
```

Arguments

`result`	Output from `strategize` with `adversarial = TRUE`
`validate`	Logical. Whether to run equilibrium validation. Default is `TRUE`. Set to `FALSE` for faster output without validation.
`check_hessian`	Logical. Whether to include Hessian geometry analysis. Default is `TRUE`. Only runs if Hessian functions are available in the result.
`verbose`	Logical. Whether to print the summary. Default is `TRUE`.

Details

This function provides a unified view of adversarial equilibrium results:

- Equilibrium vote share Q* with standard error
- Optimized strategies for both parties (AST and DAG)
- Equilibrium quality metrics (best-response errors)
- Four-quadrant contribution breakdown
- Voter proportion information
- Convergence status

Value

Invisibly returns a list containing all summary statistics.

Examples

```
## Not run:
# Run adversarial strategize
result <- strategize(Y = y, W = w, adversarial = TRUE, nSGD = 500)

# Print summary
summarize_adversarial(result)
```

```
# Quick summary without validation
summarize_adversarial(result, validate = FALSE)

## End(Not run)
```

`summary.strategize_result` *Summary Method for strategize Results*

Description

Creates a summary table comparing baseline and optimal distributions.

Usage

```
## S3 method for class 'strategize_result'
summary(object, ...)
```

Arguments

`object`	A strategize result object
`...`	Additional arguments (ignored)

Value

Invisibly returns a data.frame with the comparison

`validate_equilibrium` *Validate Nash Equilibrium Quality*

Description

Computes best-response error to verify that the optimized strategies form a Nash equilibrium. At a true Nash equilibrium, neither player can improve their payoff by unilaterally changing strategy.

Usage

```
validate_equilibrium(
  result,
  method = c("grid", "gradient"),
  resolution = 50,
  tolerance = 0.01,
  nMonte = 100,
  plot = TRUE,
  verbose = TRUE,
  seed = 1L
)
```

Arguments

`result`	Output from `strategize` with `adversarial = TRUE`
`method`	Character string specifying the search method: • `"grid"`: Grid search around the current solution (deterministic for 1-2 parameters; random sampling for higher dimensions) • `"gradient"`: Gradient ascent from current solution (faster) Default is `"grid"`.
`resolution`	Integer. Number of grid points per dimension for grid search. Default is 50.
`tolerance`	Numeric. Maximum BR error to consider as equilibrium. Default is 0.01 (i.e., neither player can improve vote share by more than 1%).
`nMonte`	Integer. Number of Monte Carlo samples for Q evaluation. This overrides the Monte Carlo settings stored in `result` for the duration of this validation call. Default is 100.
`plot`	Logical. Whether to generate visualization. Default is `TRUE`.
`verbose`	Logical. Whether to print progress messages. Default is `TRUE`.
`seed`	Integer seed for reproducible Monte Carlo evaluation and deterministic search behavior. Use `NULL` for non-deterministic behavior. Default is 1.

Details

What is a Nash Equilibrium?

In the adversarial setting, two parties (AST and DAG) simultaneously choose probability distributions over candidate attributes (e.g., gender, age, policy positions). Each party wants to maximize their expected vote share given the opponent's strategy. A Nash equilibrium is a pair of strategies where:

- AST's strategy is optimal given DAG's strategy
- DAG's strategy is optimal given AST's strategy

At equilibrium, neither party can improve by unilaterally changing their strategy.

What is Best-Response Error?

The best-response error measures how far a player's current strategy is from being optimal. For player p, it is defined as:

$$BR\_error_p = \max_{\pi_p} Q(\pi_p, \pi^*_{-p}) - Q(\pi^*_p, \pi^*_{-p})$$

In words: if we fix the opponent's strategy and search for the best possible response, how much better could we do compared to our current strategy?

- **BR error = 0**: The player is already playing optimally (true equilibrium)
- **BR error > 0**: The player could improve by switching strategies
- **BR error = 0.05**: The player could gain 5 percentage points in vote share

How Validation Works

This function performs the following steps:

1. Evaluates Q (expected vote share) at the current solution
2. For AST: fixes DAG's strategy and searches for AST's best response

3. For DAG: fixes AST's strategy and searches for DAG's best response
4. Computes the improvement each player could achieve (BR error)
5. If both BR errors are below tolerance, declares it a valid equilibrium

Interpretation

- `is_equilibrium = TRUE`: The solution is a valid Nash equilibrium (within numerical tolerance). Both parties are playing optimally.
- `is_equilibrium = FALSE`: At least one party could improve by changing strategy. This may indicate insufficient SGD iterations, a local minimum, or numerical issues.

Search Methods

- `"grid"`: Searches over a discretized grid of strategies around the current solution. More thorough but slower. Recommended for validation.
- `"gradient"`: Runs additional gradient ascent steps from the current solution. Faster but may miss improvements in other directions.

Value

A list containing:

br_error_ast Best-response error for AST player (how much AST could improve by switching to best response)

br_error_dag Best-response error for DAG player

is_equilibrium Logical. TRUE if both errors are below tolerance

Q_current Current objective value at the solution

Q_br_ast Objective value if AST switched to best response

Q_br_dag Objective value if DAG switched to best response

br_strategy_ast The best response strategy for AST (for comparison)

br_strategy_dag The best response strategy for DAG

nMonte Monte Carlo sample count used for validation

seed Seed used for deterministic evaluation (if provided)

plot ggplot object if plot=TRUE and ggplot2 available, else NULL

Examples

```
## Not run:
# Run adversarial strategize
result <- strategize(Y = y, W = w, adversarial = TRUE, nSGD = 500)

# Validate equilibrium
validation <- validate_equilibrium(result)
print(validation$is_equilibrium)
print(validation$br_error_ast)
print(validation$br_error_dag)

# If validation fails, try more SGD iterations
if (!validation$is_equilibrium) {
  result2 <- strategize(Y = y, W = w, adversarial = TRUE, nSGD = 2000)
  validation2 <- validate_equilibrium(result2)
```

```
}

## End(Not run)
```

Index